Universidade Municipal de São Caetano do Sul

APOSTILA DE ESTUDO PARA O ENADE SOBRE ENGENHARIA DE SOFTWARE

São Caetano do Sul
2026

MÓDULO 1 — Engenharia de Requisitos
1.1 Requisitos funcionais e não funcionais
1.2 Stakeholders
1.3 Princípios da Engenharia de Requisitos
1.4 Especificação, validação, mudanças e rastreabilidade

MÓDULO 2 — Elicitação e Modelagem de Requisitos
2.1 Técnicas de elicitação
2.2 UML

MÓDULO 3 — Fluxos e Modelos de Desenvolvimento
3.1 Fluxos de Software
3.2 Os principais de Modelos de Processos de Software
3.3 RUP

MÓDULO 4 — Métodos Ágeis e Gerenciamento
4.1 Princípios Ágeis
4.2 Frameworks e práticas Ágeis
4.3 Scrum e seus papéis
4.4 PMBOK

MÓDULO 5 — Orientação a Objetos, Arquitetura e Design
5.1 POO: abstração, encapsulamento, herança e polimorfismo
5.2 SOLID
5.3 Arquitetura em camadas
5.4 Cliente-servidor e microsserviços
5.5 Design Patterns e a Gang of Four (GoF)
5.6 ADR e trade-offs

MÓDULO 6 — Verificação, Validação e Testes
6.1 Verificação × validação
6.2 Testes unitários, integração, sistema e aceitação
6.3 Caixa-preta e caixa-branca
6.4 Casos de teste
6.5 Testes automatizados
6.6 Defeitos e rastreamento

MÓDULO 7 — UML e Modelagem de Sistemas
7.1 Diagrama de casos de uso
7.2 Diagrama de classes
7.3 Diagrama de sequência
7.4 Diagrama de componentes
7.5 Diagrama de implementação

MÓDULO 1 — Engenharia de Requisitos

1.1 Requisitos de software: o que são, tipos e etapas essenciais

O desenvolvimento de software é uma peça importante na estratégia de diversos negócios. Assim, um dos maiores desafios enfrentados por desenvolvedores é a definição dos requisitos de software: entender o que é essencial, quais funcionalidades devem ser priorizadas e como garantir a satisfação do usuário final.

Mas para quem decide investir em tecnologia, essa definição carrega um peso que vai além do time de desenvolvimento. Requisito mal escrito é orçamento estourado, prazo perdido e, cada vez mais, código gerado por IA que resolve exatamente o problema errado, com testes passando e usuário insatisfeito.

O que são requisitos de software?

Requisitos de software são especificações que definem as funcionalidades e restrições de um software. Eles são utilizados para guiar o processo de desenvolvimento, garantindo que o resultado atenda às necessidades levantadas durante a etapa de concepção do produto e aos objetivos do negócio.

Além disso, servem como base para todas as fases seguintes do desenvolvimento, como o protótipo e o design, a implementação e os testes.

Quando bem definidos, os requisitos de um sistema facilitam a comunicação entre os stakeholders e isso, por sua vez, permite uma gestão de expectativas mais eficiente.

Por que requisitos mal definidos custam caro

A reunião de levantamento de requisitos que parece burocrática no início de um projeto é, na verdade, o ponto mais barato para prevenir retrabalho. Ignorá-la não elimina o custo, apenas adia e multiplica. Esse cálculo fica ainda mais sensível quando o requisito ignorado é de conformidade.

Tipos de requisitos de software

Existem dois tipos principais de requisitos: funcionais (RF) e não funcionais (RNF).



Requisitos funcionais

Os requisitos de software funcionais são a base de qualquer projeto de desenvolvimento de software. São especificações detalhadas das funcionalidades que o sistema deve possuir para resolver o problema para o qual ele foi projetado.

Podemos classificar os requisitos de software funcionais de acordo com seu nível. A diferenciação mais comum é:

- Requisitos de negócio;
- Requisitos de usuário;
- Requisitos técnicos.

A clareza que esses requisitos proporcionam é o que permite uma boa comunicação, um bom planejamento e o desenvolvimento direcionado, e isso resulta em um produto mais adequado ao público e ao contexto.

Por exemplo: um aplicativo de uma instituição financeira, voltado ao cliente, deve ser capaz de realizar transações via PIX, pagar boletos e fazer investimentos. Esses são três requisitos do produto digital que permitem aos usuários a realização de tais funções.

Abaixo, alguns exemplos comuns de requisitos funcionais e suas aplicações:

- **Autenticação de usuário:** o sistema deve permitir que os usuários se registrem, façam login e logout de forma segura;

- **Gestão de inventário:** em um sistema de controle de estoque, por exemplo, deve ser possível adicionar, remover e atualizar facilmente informações sobre os produtos;

- **Processamento de pagamentos:** uma loja online deve ser capaz de processar pagamentos de forma prática e segura;

- **Geração de relatórios:** um sistema contábil deve gerar relatórios financeiros detalhados, como balanços e demonstrações de resultados.

Requisitos não funcionais

Os requisitos de software não funcionais (RNFs) definem as características, qualidades e restrições que o sistema deve ter. De forma mais objetiva, ele se refere a como o sistema se comportará e exercerá tais funcionalidades, e não exatamente o que ele fará. E isso é primordial para garantir que o software funcione corretamente, seja eficiente, seguro e fácil de usar.

Não à toa, também são chamados de atributos de qualidade. Isso quer dizer que são uma definição estruturada e categorizada daquilo que é necessário para mostrar que um produto tem qualidade. Assim como os funcionais, eles se subdividem em:

- **Requisitos de produto**: confiabilidade, eficiência, portabilidade, usabilidade, desempenho e espaço;

- **Requisitos organizacionais**: entrega, implementação e padrões;

- **Requisitos externos**: interoperabilidade, éticos, segurança, privacidade e legais.

Podemos exemplificar citando a adequação de produtos digitais à LGPD, que é um requisito não funcional muito comum. Outro exemplo é a afirmação de que um software precisa ser multi-plataforma, operando em Android e iOS. Ou, ainda, quando definimos que uma solução precisa utilizar criptografia, pois manipula dados sensíveis.

Em linhas gerais, podemos dizer que os requisitos não funcionais são atributos que melhoram a qualidade do software e a experiência do usuário. Veja exemplos:

- **Desempenho**: define a velocidade de resposta a determinadas ações como quantas transações por segundo o sistema deve ser capaz de processar;

- **Usabilidade**: refere-se à facilidade com que os usuários podem aprender a utilizar o sistema e executar tarefas;

- **Confiabilidade**: trata da capacidade do sistema de operar corretamente por um determinado período;

- **Segurança**: envolve a proteção contra acesso não autorizado e ataques. O armazenamento de senhas de forma criptografada é um exemplo;

- **Escalabilidade**: descreve a capacidade do sistema de crescer e lidar com um aumento na carga de trabalho, ou seja, qual porcentagem a mais de usuários ativos ele suporta sem perder desempenho.

Requisitos funcionais x não funcionais: a diferença direta

Quando o time e os stakeholders precisam alinhar rapidamente o que cada tipo de requisito representa, uma tabela resolve mais rápido do que um parágrafo:

Critério	Requisito funcional	Requisito não funcional
Responde a	O que o sistema faz	Como o sistema se comporta
Natureza	Ação, função, regra de negócio	Qualidade, restrição, atributo
Exemplo	Permitir login com e-mail e senha	Concluir o login em até 2 segundos
Falha visível como	O botão ou a função não funciona	O sistema funciona, mas mal: lento, inseguro ou instável
Como se valida	Teste funcional	Teste de carga, segurança ou usabilidade

Repare que os dois tipos são interdependentes. Um requisito funcional sem o não funcional correspondente entrega uma função que existe, mas que não aguenta o uso real: um PIX que processa, mas trava com 500 usuários simultâneos, por exemplo.

RNFs na prática: nossas dicas

Sob uma perspectiva mais prática, para o time que está construindo o produto, os requisitos não funcionais podem ser restrições, padrões a seguir, ou ainda exigir soluções bem específicas.

É natural, por exemplo, que eles precisem ser resolvidos no nível da arquitetura do software e não na programação, porque os RNFs são transversais, ou seja, afetam todas as funcionalidades.

Certa vez, desenvolvemos um produto que possuía um requisito não funcional muito crítico em relação à disponibilidade: a aplicação precisava operar 24×7, durante três meses, pois era uma das aplicações utilizadas na operação de um programa de televisão de reality show.

Esse tipo de requisito exige uma análise arquitetural: é preciso identificar e remover pontos de falha, criar soluções de fall-back (plano b) e de recuperação automática. O requisito afeta até mesmo o hardware e a infraestrutura, que precisam trabalhar com soluções de alta disponibilidade.

De requisito vago a requisito testável: um exemplo

A diferença entre um requisito não funcional bem escrito e um mal escrito não está na intenção, está na possibilidade de medir se ele foi cumprido. Compare:

- **Vago:** “o sistema deve ser rápido.”

- **Testável:** “95% das requisições devem responder em menos de 300 milissegundos, com 5 mil usuários simultâneos ativos.”

O primeiro formato parece suficiente em uma reunião, mas não dá ao time de desenvolvimento nenhum critério de aceite objetivo. O segundo formato pode virar teste automatizado, virar cláusula contratual e virar critério de aprovação na etapa de verificação.

É esse nível de precisão que separa um documento de requisitos decorativos de um documento que realmente protege o cronograma e o orçamento do projeto.

Quais são as 5 etapas da análise de requisitos?

Existem cinco etapas fundamentais da análise de requisitos de software. São elas:

1. **Levantamento**;
2. **Documentação**;
3. **Validação**;
4. **Gerenciamento**;
5. **Verificação e aprovação**.

Veja cada uma em detalhes.

1. Levantamento de requisitos

O levantamento de requisitos de software é uma etapa que envolve identificar e documentar as necessidades e expectativas dos stakeholders.

É importante que sejam utilizados métodos eficientes durante a coleta desses requisitos, para, então, garantir a confiabilidade e a adequação às demandas do sistema. Alguns desses métodos são:

- **Entrevistas com stakeholders**: envolve conversas diretas com pessoas envolvidas no projeto para reunir informações com maior profundidade;

- **Workshops de requisitos**: são sessões colaborativas para discutir e definir requisitos do sistema. Pode ser mais rápido do que conduzir entrevistas individuais, e facilita a comunicação entre diferentes grupos envolvidos;

- **Questionários**: são bastante utilizados para coleta de informações em grande volume, de forma estruturada e padronizada;

- **Observações empíricas**: envolve observar os usuários em seu ambiente de trabalho, ou através de ferramentas como Hotjar, por exemplo, para entender melhor seus processos e identificar necessidades não verbalizadas.



A definição dos requisitos é um processo contínuo e colaborativo que envolve diversos stakeholders, incluindo clientes, usuários finais, Team Managers, Desenvolvedores e Analistas de Negócios.

Quem define os requisitos e quando?

Depende do tipo de processo e ciclo de vida adotado, dois conceitos que também vêm da Engenharia de Software. Mas uma coisa a gente pode afirmar com tranquilidade: hoje, a construção dos requisitos de software é mais colaborativa que antes.

Você já reparou que, apesar de ser um dos conceitos centrais na Engenharia de Software, não é mais tão comum ouvirmos falar de requisitos? Isso acontece porque eles acabam associados a um modelo de trabalho cascata (Waterfall), burocrático e preditivo.

Em um ciclo de vida mais antigo, era normal que os requisitos fossem definidos todos no início do projeto. Era comum usar termos ‘coletar’ ou ‘elicitar’. Ou seja: a ideia era que um Analista de Sistemas ou de Negócios definiria requisitos dos usuários.

Atualmente, partimos de pressupostos um pouco diferentes. Entendemos que, para produtos inovadores, é impossível descobrir todos os requisitos no início do desenvolvimento.

Por esse motivo, a construção dos requisitos precisa ser colaborativa. A configuração varia muito, mas podemos ter:

- **Product Manager (PM)** atuando em nível de negócio, definindo visão de produto, métricas e objetivos;

- **Product Owner (PO)**, em contato direto com o time, facilitando o processo de transformar os grandes objetivos em histórias de usuário;

- **Time de desenvolvimento**, que participa desse refinamento, executa testes e análises para identificar feedbacks que contribuem para a melhoria contínua do produto; entre outros envolvidos.

Por fim, é importante lembrar que a coleta das necessidades e expectativas dos stakeholders deve ocorrer desde o início da concepção da solução digital e continuar durante todo o ciclo de desenvolvimento. Assim, é possível fazer ajustes conforme necessário.

2. Documentação de requisitos

Documentar requisitos de software de forma clara e precisa é essencial para garantir o sucesso de qualquer produto. Afinal, a documentação detalhada serve como um guia para todas as etapas do desenvolvimento.

Para isso, algumas ferramentas e técnicas são bem úteis. Aqui estão as mais comuns:

- **Documentação de especificação**: são documentos textuais detalhados que descrevem os requisitos funcionais e não funcionais do sistema. Como exemplo, temos o documento de requisitos de software (SRS) e o documento de especificação funcional (FSD);

- **Diagramas**: ajudam a visualizar os requisitos e suas interações de forma gráfica, o que facilita o entendimento e a comunicação;

- **Modelos**: são representações do sistema que ajudam a descrever e analisar seus aspectos funcionais e não funcionais. Técnicas de modelagem comuns incluem a modelagem de processos de negócio e a modelagem de dados.

3. Validação de requisitos

A validação de requisitos de software envolve a confirmação de que as especificações documentadas estão corretas, completas e compreendidas por todos os envolvidos.

As técnicas que podem ser usadas para essa validação são:

- **Revisões**: envolvem a análise dos requisitos por uma equipe de stakeholders e especialistas, por exemplo, os Tech Leads, garantindo que todos os aspectos sejam confirmados e aceitos;

- **Prototipagem**: consiste em criar modelos simplificados do sistema, que permitem aos usuários interagir e fornecer feedback antes da construção completa do produto. Figma e Adobe XD são ferramentas bastante usadas para isso;

- **Simulações**: utilizam representações virtuais do sistema para prever como ele se comporta em diferentes cenários.

4. Gerenciamento de requisitos

Fazer um bom gerenciamento de requisitos de software inclui o controle das mudanças, onde qualquer alteração é registrada, avaliada e aprovada antes de ser adotada.

Para facilitar esse processo, três ferramentas dominam o mercado, cada uma com um encaixe diferente:

- **Jira:** padrão de mercado para squads que já trabalham em Scrum ou Kanban, com rastreabilidade forte entre requisito, tarefa e teste;

- **Azure DevOps:** encaixe natural para empresas com stack Microsoft, integra requisitos, repositório de código e pipelines de CI/CD no mesmo ambiente;

- **IBM DOORS:** usado em projetos de grande porte e setores regulados, como financeiro e saúde, onde a rastreabilidade formal entre requisito e norma é exigida por auditoria.

Nenhuma das três substitui o processo. Elas só falham menos silenciosamente do que uma planilha quando o volume de requisitos cresce.

5. Verificação e aprovação

A verificação e aprovação de requisitos de software envolvem uma revisão final e a aprovação formal pelos stakeholders.

É uma etapa fundamental, já que garante que todos os requisitos estão corretos, completos e alinhados com as expectativas antes de iniciar o desenvolvimento, evitando retrabalhos, desperdícios e garantindo que o produto digital cumpra seus objetivos.

Por que requisitos funcionais não podem faltar no seu projeto?

Quando os requisitos ficam mal definidos ou incompletos, a equipe de desenvolvimento entra em modo de retrabalho contínuo. Retrabalho é o maior ladrão de margem de lucro em projetos de TI e a causa número um de softwares abandonados no meio do caminho. Os requisitos funcionais não podem ser tratados como burocracia: eles são o ativo mais valioso da fase de planejamento.

Como documentar requisitos funcionais e não funcionais na prática?

Não basta ter a ideia na cabeça. É preciso documentar. Uma software house profissional segue etapas rigorosas para garantir que nenhum requisito seja esquecido.

- **Concepção:** entender a visão macro do negócio e onde o cliente quer chegar.

- **Elicitação:** entrevistar usuários e stakeholders para descobrir o que eles realmente precisam, aqui nascem os requisitos funcionais e os requisitos funcionais e não funcionais que sustentam o produto.

- **Elaboração:** criar diagramas, fluxos e user stories.

- **Negociação:** definir prioridades e o que entra no MVP (Produto Mínimo Viável).

- **Especificação:** traduzir os pedidos em documentos técnicos e cláusulas contratuais.

- **Validação:** o cliente revisa e assina o escopo.

- **Gerenciamento:** monitorar, durante todo o ciclo, se o que está sendo construído segue o combinado.

1.2 Stakeholders e seu Impacto: Entendendo sua Influência nos Projetos

Os stakeholders são uma parte essencial de qualquer projeto. Eles representam as partes interessadas que podem impactar ou serem impactadas pelos resultados das atividades desenvolvidas. Entender como gerenciá-los e engajá-los é crucial para o sucesso de qualquer iniciativa. Se você é um empreendedor ou um gerente de projetos, reconhecer a influência das partes interessadas e aprender as suas expectativas pode transformar seus resultados e aumentar significativamente as chances de sucesso.

O que são Stakeholders?

O termo stakeholders refere-se a indivíduos, grupos ou organizações que têm interesse em um projeto e seus resultados. Eles podem ser classificados em diversas categorias, como:

- **Stakeholders internos:** funcionários, gerentes e diretores envolvidos diretamente no projeto.

- **Stakeholders externos:** clientes, fornecedores, acionistas e a comunidade em geral.

- **Stakeholders primários:** aqueles que têm uma influência direta sobre os resultados do projeto.

- **Stakeholders secundários:** aqueles que têm um interesse indireto.

A correta identificação dos stakeholders é um primeiro passo crucial para a gerência de suas expectativas e influências.

Por que os Stakeholders são Importantes?

A influência de partes interessadas nos projetos não pode ser subestimada. Eles podem;

- Oferecer informações valiosas durante o desenvolvimento do projeto;
- Ajudar a identificar riscos e oportunidades;
- Influenciar a aceitação dos resultados finais;
- Possui a capacidade de dar suporte financeiro e operacional.

Se bem gerenciados, os stakeholders podem se tornar aliados poderosos, contribuindo para o sucesso de um projeto. Portanto, entendê-los e saber como envolver e gerenciar suas expectativas é vital.

Identificando Stakeholders: Um Passo Crucial

Uma das primeiras etapas na gestão de stakeholders é a identificação. Para isso, considere as seguintes abordagens:

- **Mapeamento de stakeholders:** crie um diagrama que colecione e classifique todos os stakeholders relevantes para seu projeto.

- **Análise de influência:** classifique os stakeholders com base em seu nível de influência e interesse no projeto.

- **Entrevistas e feedback:** converse diretamente com os stakeholders para entender suas expectativas e preocupações.

Essas estratégias ajudarão a construir uma base sólida para a sua equipe de gestão.

Impacto dos Stakeholders nos Resultados do Projeto

Os stakeholders têm um impacto significativo nos resultados de um projeto. Um feedback negativo ou uma insatisfação por parte de uma parte interessada chave pode resultar em atrasos, aumento de custos ou até mesmo o fracasso do projeto. Da mesma forma, o engajamento positivo pode levar a resultados fenomenais.

1.3 Princípios da Engenharia de Requisitos

A Engenharia de Requisitos (ER) é a disciplina sistemática de elicitação, análise, especificação, validação e gerenciamento de requisitos de sistemas, com o objetivo de garantir que o produto final atenda às necessidades e expectativas dos stakeholders. Seus princípios fundamentais constituem a base conceitual que orienta todas as atividades e práticas da área, independentemente do processo de desenvolvimento adotado (tradicional, ágil ou híbrido).

Abaixo são descritos detalhadamente os nove princípios fundamentais definidos pelo *International Requirements Engineering Board* (IREB) no guia de certificação CPRE (Certified Professional for Requirements Engineering), que é a referência global mais consolidada na área, complementados por perspectivas de autores consagrados da literatura.

Os 9 Princípios Fundamentais da Engenharia de Requisitos

1. Orientação para o valor: Os requisitos são um meio para um fim, não um fim em si mesmo.

O ato de definir e documentar requisitos não é um objetivo por si só. Os requisitos só são úteis se agregarem valor ao projeto e aos stakeholders. O valor de um requisito é definido como seu benefício menos seu custo:

- **Benefício:** grau em que o requisito contribui para o sucesso do sistema, satisfazendo necessidades e reduzindo riscos de falha ou retrabalho.

- **Custo:** esforço para elicitar, validar, documentar e gerenciar o requisito.

Detectar e corrigir um requisito errado ou ausente durante a implementação ou operação pode custar de 10 a 100 vezes mais do que especificá-lo corretamente desde o início. Portanto, cada requisito deve ser avaliado quanto ao seu valor real, evitando o investimento em funcionalidades que não entregam benefícios significativos.

2. Stakeholders: A ER é sobre satisfazer os desejos e necessidades dos stakeholders.

O objetivo final de qualquer sistema é resolver problemas e atender expectativas de pessoas e organizações que têm interesse no sistema, os stakeholders. Estes incluem não apenas usuários finais e clientes, mas também desenvolvedores, operadores, reguladores, acionistas e qualquer pessoa ou grupo que possa ser afetado pelo sistema.

Cada stakeholder tem um papel e uma perspectiva diferente: usuários buscam usabilidade, gerentes buscam custo-benefício, equipes de segurança buscam proteção, etc. É fundamental:

- Identificar todos os stakeholders relevantes

- Classificá-los por grau de influência no sucesso do sistema (alta, média, baixa)

- Reconhecer e resolver conflitos entre suas necessidades, por meio de consenso, negociação ou especificação de variantes do sistema

Não considerar stakeholders essenciais é uma das principais causas de falha em projetos de software.

3. Entendimento comum: O desenvolvimento bem-sucedido é impossível sem uma base de compreensão compartilhada

A ER tem como função central criar, fomentar e assegurar um entendimento comum entre todos os envolvidos: stakeholders, engenheiros de requisitos e desenvolvedores. Existem duas formas de entendimento compartilhado:

- **Explícito:** alcançado por meio de requisitos cuidadosamente elaborados, documentados e acordados (característico de processos orientados por planejamento).

- **Implícito:** baseado na confiança e no conhecimento compartilhado de necessidades e contexto (comum em processos ágeis).

O risco do "falso entendimento" quando as pessoas acreditam compartilhar a mesma visão, mas na verdade interpretam os requisitos de maneiras diferentes é um dos maiores desafios da ER. Por isso, o entendimento compartilhado deve ser continuamente verificado e assegurado, concentrando-se nos aspectos relevantes dentro dos limites do sistema e seu contexto.

4. Contexto: Sistemas não podem ser compreendidos isoladamente

Nenhum sistema existe no vácuo. Todo sistema opera dentro de um contexto específico que inclui:

- Ambiente organizacional (estrutura, processos, cultura)

- Ambiente técnico (infraestrutura, sistemas existentes, tecnologias)

- Ambiente social e legal (normas, regulamentações, políticas)

- Limites do sistema (o que está dentro e fora do escopo)

A ER deve ir além da definição de requisitos internos ao sistema e considerar também:

- Como mudanças no contexto podem afetar os requisitos

- Quais requisitos do mundo real são relevantes para o sistema

- Como mapear adequadamente requisitos do mundo real para requisitos do sistema

- Quais suposições sobre o contexto devem ser garantidas para o funcionamento correto do sistema

Ignorar o contexto é um erro frequente que leva a sistemas que funcionam tecnicamente, mas falham em se integrar ao ambiente onde devem operar.

5. Problema, requisito e solução: Uma trinca inevitavelmente entrelaçada

Problemas, requisitos e soluções estão intrinsecamente ligados e influenciam-se mutuamente. Toda situação de insatisfação com o estado atual pode ser vista como um problema; os requisitos definem o que o sistema deve fazer para resolvê-lo; e a solução é a implementação concreta que atende aos requisitos.

Esses três elementos não seguem necessariamente uma ordem linear: em sistemas inovadores, ideias de solução podem gerar novos requisitos e até redefinir o problema original. O entrelaçamento se manifesta de várias formas:

- **Hierárquico:** requisitos de alto nível levam a decisões de projeto que geram requisitos de nível inferior

- **Viabilidade técnica:** só é possível avaliar se um requisito é viável ao explorar soluções técnicas

- **Validação:** protótipos (soluções parciais) são usados para validar requisitos

- **Viés de solução:** diferentes stakeholders podem propor soluções diferentes para o mesmo problema, gerando requisitos conflitantes

Embora o entrelaçamento seja inevitável, bons engenheiros de requisitos esforçam-se para separar preocupações com problemas, requisitos e soluções ao pensar, comunicar e documentar, para facilitar a análise e evitar que soluções pré-concebidas limitem a compreensão do problema real.

6. Validação: Requisitos não validados são inúteis

A validação é o processo de confirmar que os requisitos documentados correspondem realmente às necessidades dos stakeholders ou seja, garantir que foram especificados os requisitos corretos. Não há especificação de requisitos sem validação.

Validar requisitos significa verificar se:

- Foi alcançado acordo entre os stakeholders (conflitos resolvidos, prioridades definidas)

- Os desejos e necessidades dos stakeholders são devidamente cobertos

- As suposições sobre o domínio e o contexto são razoáveis

Realizar a validação apenas no final do desenvolvimento é extremamente arriscado e custoso. A validação deve começar já durante a própria ER, por meio de técnicas como revisões, inspeções, prototipagem e validação com stakeholders representativos.

7. Evolução: A mudança de requisitos não é um acidente, mas o caso normal

Todo sistema técnico está sujeito a evolução. Necessidades de negócio, ambiente competitivo, tecnologia e expectativas dos usuários mudam continuamente e, como consequência, os requisitos também mudam. Ignorar essa realidade é inútil e prejudicial.

As mudanças podem ocorrer por diversos motivos:

- Processos de negócio alterados
- Novos produtos ou serviços de concorrentes
- Mudanças de prioridade ou opinião dos clientes
- Avanços tecnológicos
- Feedback de usuários
- Detecção de erros ou suposições defeituosas nos requisitos

Os engenheiros de requisitos devem equilibrar dois objetivos aparentemente contraditórios:

- Permitir que os requisitos sejam alterados, pois a evolução é inevitável

- Manter certa estabilidade, pois mudanças diárias tornam o desenvolvimento impossível e custoso

Sem um processo sistemático de gerenciamento de mudanças, a evolução dos requisitos acabará gerenciando a equipe, e não o contrário.

8. Inovação: Mais do mesmo não é mais o suficiente

Bons engenheiros de requisitos não são meros "gravadores de voz" que registram passivamente o que os stakeholders pedem. Dar aos stakeholders exatamente o que eles pedem significa perder a oportunidade de fazer as coisas melhor do que antes.

A inovação na ER pode ocorrer em dois níveis:

- **Pequena escala:** buscar novas funcionalidades, simplicidade de uso e soluções mais inteligentes para os problemas apresentados. Por exemplo, em vez de simplesmente implementar botões de zoom em um relatório que não cabe na tela, analisar se colunas podem ser eliminadas ou colapsadas, ou se a interação pode ser otimizada para o dispositivo (tablet, por exemplo).

- **Grande escala:** explorar com os stakeholders formas disruptivas de fazer as coisas, que podem gerar inovação massiva e vantagem competitiva.

Bons engenheiros de requisitos buscam não apenas satisfazer os stakeholders, mas encantá-los, ao mesmo tempo que evitam a arrogância de acreditar que sabem tudo melhor do que eles.

9. Trabalho sistemático e disciplinado: Não podemos prescindir dele na ER

A Engenharia de Requisitos não é uma arte, mas uma disciplina que exige trabalho sistemático e rigoroso. Independentemente do processo de desenvolvimento adotado, é necessário empregar práticas adequadas para elicitar, documentar, validar e gerenciar requisitos de forma consistente.

Trabalho sistemático e disciplinado significa:

- Configurar um processo de ER adaptado ao problema específico e ao processo de desenvolvimento do sistema

- Selecionar práticas e produtos de trabalho adequados ao contexto, ao problema e ao ambiente de trabalho

- Não usar sempre o mesmo processo ou as mesmas práticas de forma acrítica

- Não reutilizar cegamente processos e práticas que funcionaram em projetos anteriores, sem avaliar se são adequados à nova situação

Agilidade e flexibilidade não são desculpas para um trabalho desordenado e ad hoc na ER. Não existe um "tamanho único" que funcione para todos os projetos; a adaptação é fundamental.

1.4 Especificação, Validação, Mudanças e Rastreabilidade na Engenharia de Requisitos

A Engenharia de Requisitos é a fase mais crítica do desenvolvimento de software. Erros cometidos aqui se propagam por todo o ciclo de vida do projeto, tornando-se exponencialmente mais caros para corrigir na fase de codificação ou testes.

Para garantir que a equipe construa o produto certo e atenda às reais necessidades dos stakeholders, a Engenharia de Requisitos baseia-se em processos fundamentais. Abaixo está o detalhamento de quatro pilares essenciais: **Especificação, Validação, Gerenciamento de Mudanças e Rastreabilidade**.

1. Especificação de Requisitos

A especificação é o processo de traduzir as necessidades e expectativas (frequentemente vagas ou conflitantes) coletadas dos stakeholders em um documento formal, estruturado e compreensível tanto para o cliente quanto para a equipe técnica.

- **O que é gerado:** O artefato clássico é o **Documento de Especificação de Requisitos de Software (SRS - Software Requirements Specification)**. Em metodologias ágeis, isso se traduz no **Product Backlog** (composto por Épicos e Histórias de Usuário).

- **Características de uma boa especificação:**

    - **Clareza e Ausência de Ambiguidade:** Deve haver apenas uma interpretação possível.

    - **Completude:** Todas as funcionalidades e restrições necessárias devem estar descritas.

    - **Consistência:** Um requisito não pode contradizer o outro.

    - **Testabilidade:** Deve ser possível verificar se o requisito foi atendido no produto final.

- **Técnicas utilizadas:** Linguagem natural estruturada, Casos de Uso (UML), Histórias de Usuário com Critérios de Aceite, e, em sistemas críticos, métodos formais (matemáticos).

2. Validação de Requisitos

Enquanto a especificação garante que escrevemos os requisitos corretamente, a **validação** garante que escrevemos os requisitos *certos*. É o momento de perguntar:

"Este sistema, se construído exatamente como especificado, resolverá o problema de negócio do cliente?"

- **O Risco:** Construir um software tecnicamente perfeito, mas que não tem utilidade para o usuário final.

- **Técnicas de Validação:**

    - **Revisões de Requisitos (Walkthroughs):** Reuniões estruturadas onde a equipe e o cliente leem a especificação juntos para encontrar erros, omissões ou conflitos.

    - **Prototipagem:** Criação de interfaces visuais (wireframes ou mockups interativos) para que o usuário "sinta" o sistema antes de ele ser programado. É a técnica mais eficaz para descobrir requisitos não mapeados.

    - **Geração de Casos de Teste:** Escrever testes baseados nos requisitos antes de codificar. Se um requisito for muito difícil de testar, ele provavelmente está mal especificado.

3. Gerenciamento de Mudanças

No mundo real, os requisitos mudam. O mercado muda, leis são alteradas, ou o cliente simplesmente entende melhor o que quer após ver as primeiras entregas. O Gerenciamento de Mudanças é o processo de controlar essas alterações de forma que não destruam o cronograma, o orçamento ou a arquitetura do projeto.

- **O Processo Tradicional (CCB - Change Control Board):**

    1. **Solicitação:** O cliente ou membro da equipe propõe uma mudança.

    2. **Análise de Impacto:** Avalia-se quais outros requisitos, módulos de código, cronogramas e custos serão afetados.

    3. **Aprovação/Rejeição:** Um comitê decide se a mudança será incorporada.

4. **Atualização:** A documentação e os planos são atualizados.

- **Perspectiva Ágil:** A mudança não é vista como um "erro" a ser evitado, mas como uma vantagem competitiva. O gerenciamento de mudanças é feito continuamente através do refinamento (grooming) do Product Backlog e da repriorização a cada nova Sprint.

4. Rastreabilidade de Requisitos

A rastreabilidade é a capacidade de acompanhar a vida de um requisito em ambas as direções (para frente e para trás), desde a sua origem até a sua implementação em código e validação em testes.

- **Rastreabilidade para Trás (Backward):** Conecta o requisito à sua origem. *Por que este requisito existe? Quem o pediu? Qual regra de negócio ele atende?*

- **Rastreabilidade para Frente (Forward):** Conecta o requisito aos artefatos técnicos de design, código e casos de teste. *Onde este requisito foi implementado? Como sabemos que ele funciona?*

- **Matriz de Rastreabilidade:** É a ferramenta mais comum. Trata-se de uma tabela (geralmente gerida por softwares como Jira, Azure DevOps, ou mesmo planilhas) que cruza os IDs dos requisitos com os IDs dos módulos de arquitetura, commits de código e cenários de teste.

- **Benefícios:**

    - Facilita imensamente a **Análise de Impacto** quando ocorre uma mudança.

    - Garante que nenhum requisito foi esquecido (se há um requisito sem teste ou código vinculado, ele foi esquecido).

    - Evita "Gold Plating" (código existente que não está vinculado a nenhum requisito de negócio).

MÓDULO 2 — Elicitação e Modelagem de Requisitos

2.1 Técnicas de Elicitação de Requisitos

Introdução

No mundo da engenharia de software, a excelência reside na capacidade de compreender as necessidades e expectativas dos *stakeholders* (partes interessadas), traduzindo-as em requisitos bem definidos que servirão como base para o desenvolvimento de sistemas de software bem-sucedidos. Este processo crítico, conhecido como **elicitação ou levantamento de requisitos**, é o alicerce de qualquer projeto de software e desempenha um papel essencial na ponte entre a visão inicial do sistema e sua implementação prática.

A elicitação de requisitos é uma disciplina complexa que requer uma compreensão profunda das técnicas disponíveis para coletar, analisar e documentar os requisitos de software.

Veremos desde as técnicas tradicionais, como entrevistas e questionários, até abordagens mais inovadoras, como a prototipação e a etnografia. A banca não costuma aprofundar muito nesse assunto, sendo suficiente conhecer o conceito de cada técnica e a categoria em que se encontra.

Principais Técnicas de Elicitação ou Levantamento de Requisitos

Para tornar a elicitação de requisitos mais eficiente, foram desenvolvidas diversas técnicas que podem ser usadas em conjunto. A escolha da melhor técnica depende do tempo disponível, dos recursos, da complexidade do projeto e das informações a serem obtidas. Algumas categorias de técnicas de elicitação incluem:

- Técnicas Tradicionais
- Técnicas de Elicitação de Grupo
- Prototipação
- Técnicas Cognitivas
- Técnicas Contextuais

A seguir, veremos a descrição de cada uma dessas categorias e suas principais técnicas de elicitação de requisitos.

Técnicas Tradicionais

São as técnicas mais comuns, aplicadas em várias áreas de conhecimento. Alguns exemplos são:

- **Entrevistas**: Realizar entrevistas com os stakeholders para discutir suas necessidades e desejos.

- **Questionários**: Criar questionários estruturados para coletar informações de um grande número de pessoas de forma padronizada.

- **Observação**: Observar as atividades dos usuários em seu ambiente de trabalho para entender melhor suas necessidades.

Técnicas de Elicitação de Grupo

Incentivam a colaboração e a geração de ideias entre as partes interessadas. Como exemplos temos:

- ***Workshops***: Reuniões de grupos de stakeholders para discutir e definir requisitos de forma colaborativa.

- ***Brainstorming***: Também conhecido como "**tempestade de ideias**", nesta técnica as partes interessadas são reunidas em um local e todas as ideias são declaradas em voz alta e escritas. É uma técnica amplamente utilizada para gerar um grande número de ideias, soluções ou conceitos em um ambiente de colaboração. O *brainstorming* é eficaz porque cria um ambiente que encoraja a livre expressão de ideias, **estimulando a criatividade e a colaboração**. Ele é frequentemente usado no início de um projeto para reunir uma variedade de perspectivas e explorar diferentes abordagens para um problema. Aqui estão os passos básicos e a explicação de como o brainstorming funciona:

    - **Preparação**: Reúna um grupo de pessoas com conhecimento relevante sobre o problema ou requisitos em questão.

    - **Fase de Geração de Ideias**: Durante esta fase, o grupo se reúne para discutir o tópico em questão. A principal regra é que não há julgamento de ideias nesta fase. Os participantes são incentivados a compartilhar qualquer ideia que tenham, não importa quão estranha ou improvável possa parecer. A ênfase está na quantidade de ideias, não na qualidade.

    - **Duração Limitada**: A sessão de brainstorming geralmente é cronometrada para evitar que as discussões se arrastem por muito tempo em uma única ideia.

    - **Variedade de Ideias**: Durante a sessão, as ideias são anotadas de forma visível, seja em um quadro branco, em notas adesivas ou em uma ferramenta digital. Isso permite que todos vejam as ideias sendo geradas e oferece inspiração para ideias adicionais.

    - **Discussão e Seleção**: Após a sessão de geração de ideias, o grupo revisa as ideias coletadas. Neste ponto, as ideias são debatidas e classificadas com base em sua relevância, viabilidade e valor para o projeto.

    - **Documentação**: As ideias selecionadas são documentadas e se tornam parte do conjunto de requisitos ou soluções a serem considerados no projeto.

- **JAD (*Joint Application Development*)**: É uma técnica de elicitação de requisitos que se destina a reunir, de maneira **colaborativa e intensiva**, os stakeholders **chave** de um projeto de software para definir e documentar requisitos de forma eficaz. A JAD é especialmente útil quando se deseja acelerar o processo de coleta de requisitos, melhorar a comunicação entre os envolvidos e minimizar mal-entendidos. Os principais aspectos da técnica JAD são:

    - **Reúne os participantes-chave**: A JAD reúne um grupo selecionado de participantes-chave, incluindo usuários finais, gerentes, analistas de sistemas e desenvolvedores.

    - **Sessões de trabalho intensivas**: As sessões JAD são geralmente realizadas em períodos concentrados, como vários dias, com a intenção de maximizar a produtividade.

    - **Presença de um facilitador**: Um facilitador experiente é essencial na condução das sessões JAD. O facilitador orienta o grupo, mantém o foco nas metas, ajuda a resolver conflitos e garante que as discussões sejam produtivas.

    - **Agenda bem definida**: Cada sessão JAD tem uma agenda claramente definida, que pode incluir revisão de processos, discussões sobre requisitos, revisão de protótipos e documentação.

    - **Ferramentas de apoio**: Durante as sessões JAD, são frequentemente utilizadas ferramentas de apoio, como diagramas, fluxogramas, protótipos de software e modelos conceituais, para ajudar a visualizar as ideias e conceitos.

    - **Documentação em tempo real**: À medida que os requisitos são discutidos e definidos, eles são documentados em tempo real. Isso ajuda a manter um registro das decisões tomadas e a criar um conjunto claro de requisitos.

    - **Aprovação dos stakeholder**s: No final das sessões JAD, os requisitos documentados são revisados e aprovados pelos stakeholders presentes, garantindo que haja consenso sobre o que foi acordado.

- **RAD (*Rapid Application Development*)**: é uma abordagem que se concentra na entrega rápida de sistemas de software funcionais. Utiliza técnicas como **JAD, ferramentas CASE e prototipação** para acelerar o processo de desenvolvimento e entregar um produto final em menos tempo.

Prototipação

A ideia da prototipação é criar **protótipos ou modelos** para permitir que as partes interessadas visualizem o sistema e consigam entender melhor os requisitos. Esta técnica é muito útil quando existe um alto grau de incerteza e também quando se deseja um feedback rápido dos usuários.

Técnicas Cognitivas

Técnicas cognitivas de elicitação de requisitos são um conjunto de métodos que têm como objetivo entender o pensamento e o conhecimento dos usuários, a fim de coletar informações relevantes para o desenvolvimento de sistemas de software. Essas técnicas são especialmente úteis para projetos que envolvem sistemas baseados em conhecimento, onde a compreensão das estruturas cognitivas dos usuários é fundamental. Algumas técnicas cognitivas são:

- **Análise de Protocolo**: Nessa técnica, os pesquisadores observam e registram as ações e pensamentos dos usuários enquanto eles realizam tarefas específicas. Isso envolve a coleta de dados “em tempo real” sobre como os usuários abordam um problema ou realizam uma tarefa. A análise de protocolo ajuda a identificar os padrões de pensamento, os desafios e as estratégias dos usuários, fornecendo insights valiosos para o design do sistema.

- ***Laddering***: O *laddering* é uma técnica usada para explorar as relações hierárquicas entre os valores, crenças e objetivos dos usuários. Os pesquisadores fazem perguntas abertas, como “*Por que você acha isso importante?*” ou “*Como isso se relaciona com seus objetivos?*” para aprofundar a compreensão das motivações subjacentes dos usuários. O resultado é uma representação em forma de “escada” que ilustra as conexões entre os elementos cognitivos.

- ***Card Sorting***: O *card sorting* é uma técnica que envolve a categorização e a organização de informações ou conceitos. Os usuários são convidados a agrupar cartões com informações relacionadas de acordo com seu entendimento. Essa técnica ajuda a determinar como os usuários percebem e

organizam informações, o que é útil na estruturação de interfaces de usuário e arquitetura de informação.

- ***Repertory Grids***: Os *repertory grids* são usados para explorar as percepções e o entendimento dos usuários em relação a um determinado tópico. Os usuários são solicitados a comparar e contrastar elementos (por exemplo, características de um sistema) e a descrever suas semelhanças e diferenças. Isso ajuda a identificar as dimensões que os usuários consideram mais relevantes e a entender como eles conceitualizam o tópico.

Técnicas Contextuais

Por fim, temos as técnicas contextuais, que são abordagens que se concentram na compreensão profunda do contexto em que um sistema será utilizado. Elas visam capturar as nuances do ambiente, interações sociais, práticas de trabalho e comportamento dos usuários para informar o processo de elicitação de requisitos. Algumas técnicas contextuais são:

- **Etnografia**: é uma técnica de pesquisa que envolve a imersão profunda no ambiente dos usuários e partes interessadas. Ela é originária da antropologia, mas é frequentemente usada em contextos de desenvolvimento de software para entender como as pessoas interagem com sistemas e tecnologia em seu ambiente natural. Para realizar a etnografia, os pesquisadores passam tempo observando e interagindo com os usuários em seu ambiente de trabalho ou em suas atividades cotidianas. Eles fazem anotações detalhadas, entrevistam os usuários e documentam suas experiências. A etnografia busca obter uma visão profunda da cultura, práticas e desafios do ambiente do usuário. Isso ajuda a identificar requisitos que podem não ser evidentes apenas através de entrevistas formais.

- **Análise Social**: é uma técnica que se concentra nas interações sociais e comunicações entre os stakeholders. Ela visa entender como as pessoas se comunicam, compartilham informações, tomam decisões e colaboram em um ambiente específico. A análise social envolve a observação e análise de como as pessoas se relacionam e compartilham conhecimento. Além disso, pode incluir o estudo de redes sociais, a análise de comunicações por email, reuniões e outras formas de interação. A análise social ajuda a identificar relações, hierarquias e padrões de comunicação entre os stakeholders. Isso é útil para entender como a informação flui e como as decisões são tomadas no contexto do projeto.

Síntese das Técnicas de Elicitação de Requisitos

Abaixo, apresentamos um resumo das técnicas de elicitação de requisitos agrupadas por categoria:

- **Técnicas Tradicionais**: forma mais comum de coletar informações dos stakeholders. Como exemplos temos: *entrevistas, questionários e observações*.

- **Técnicas de Elicitação de Grupo**: incentivam a colaboração e a geração de ideias entre as partes interessadas. Ex: *workshops, brainstorming, JAD e RAD*.

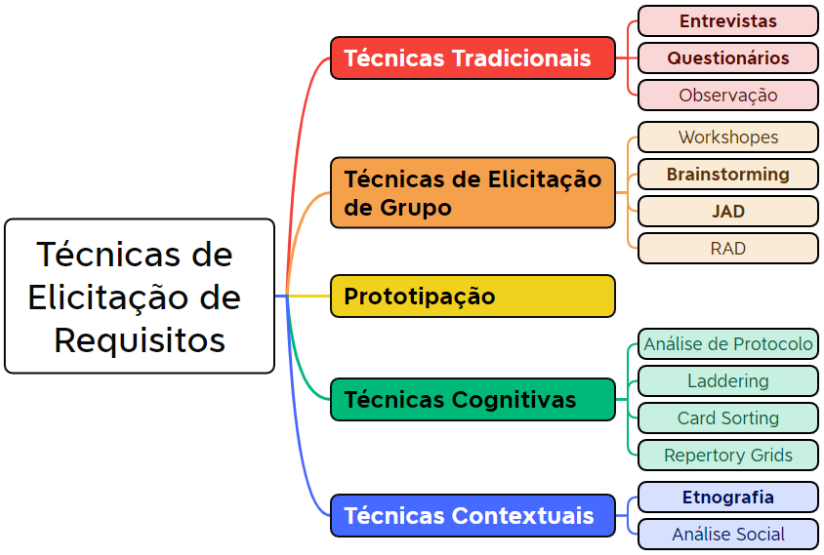
- **Prototipação**: envolve a criação de protótipos para ajudar as partes interessadas a visualizarem o sistema e obter feedback rápido.

- **Técnicas Cognitivas**: buscam entender o pensamento e o conhecimento dos usuários. Ex: *análise de protocolo, laddering, card sorting e repertory grids*.

- **Técnicas Contextuais**: focam na compreensão profunda do ambiente em que o sistema será utilizado, considerando aspectos culturais, práticas de trabalho e interações sociais. Ex: *etnografia e análise social*.

Mapa Mental das Técnicas de Elicitação de Requisitos

Por fim, apresentamos um mapa mental para facilitar a memorização e organização do assunto:



2.2 UML

A Unified Modeling Language (UML) foi criada para construir uma linguagem de modelagem visual comum, semântica e sintaticamente rica para a arquitetura, design e implementação de sistemas de software complexos, tanto estrutural quanto comportamental.

Ela é análoga às plantas baixas usadas em outras áreas e consiste em diferentes tipos de diagramas. No geral, os diagramas UML descrevem o limite, a estrutura e o comportamento do sistema e dos objetos dentro dele. Ao escrever código, há milhares de linhas em um aplicativo e é difícil acompanhar os relacionamentos e as hierarquias dentro de um sistema de software. Os diagramas UML dividem esse sistema de software em componentes e subcomponentes.

Existem quatro categorias de modelos de resolução de problemas: linguagens imperativas, funcionais, declarativas e orientadas a objetos. Nas linguagens orientadas a objetos, os algoritmos são expressos definindo-se “objetos” e fazendo com que eles interajam entre si. Esses objetos são elementos a serem manipulados e existem no mundo real. Podem ser edifícios, widgets em uma área de trabalho ou seres humanos.

As linguagens orientadas a objetos dominam o mundo da programação porque modelam objetos do mundo real. A UML é uma combinação de várias notações orientadas a objetos: design orientado a objetos, técnica de modelagem de objetos e engenharia de software orientada a objetos.

A UML utiliza os pontos fortes dessas três abordagens para apresentar uma metodologia mais consistente e mais fácil de usar. A UML representa as melhores práticas para a criação e documentação de diferentes aspectos da modelagem de sistemas de software e de negócios. Como a UML é padronizada, ela pode ser aplicada em várias linguagens de programação e processos de desenvolvimento, tornando-se amplamente legível para a maioria dos desenvolvedores de software.

Na prática, a UML também é uma forma de documentação visual que pode reduzir o tempo gasto explicando um sistema repetidamente. Essa documentação pode ajudar engenheiros de várias maneiras, desde a integração de novos funcionários até o fornecimento de uma visão geral rápida do sistema para as partes interessadas, para que os desenvolvedores não percam tempo em reuniões.

Tipos de diagramas UML

A UML usa elementos e os associa de diferentes maneiras para formar diagramas estáticos (que representam aspectos estruturais de um sistema) e diagramas comportamentais (que representam aspectos dinâmicos de um sistema). Para quem não está familiarizado, pode parecer que existe um número infinito de diagramas UML, mas, na verdade, os padrões da UML identificam 14 tipos de diagramas que são divididos em dois grupos, definidos abaixo.

Diagramas UML estruturais

- **Diagrama de classes:** este é o diagrama UML mais comumente usado e a base principal de qualquer solução orientada a objetos. Ele mostra as classes dentro de um sistema, atributos e operações, e o relacionamento entre cada classe. As classes são agrupadas para criar diagramas de classes ao diagramar sistemas grandes. Uma classe é representada por um retângulo que contém três compartimentos empilhados verticalmente: o compartimento superior contém o nome da classe e é obrigatório, mas os dois compartimentos inferiores fornecem detalhes sobre os atributos da classe e as operações ou comportamentos da classe.

- **Diagrama de componentes:** um diagrama de componentes é essencialmente uma versão mais especializada do diagrama de classes, as mesmas regras de notação se aplicam a ambos. Um diagrama de componentes divide um sistema complexo em componentes menores e visualiza o relacionamento entre eles.

- **Diagrama de estrutura composta:** esses tipos de diagramas são essencialmente as plantas para a estrutura interna de um classificador. Eles também podem ser usados para mostrar o comportamento de uma colaboração ou as interações do classificador com seu ambiente por meio de portas. Eles podem descrever facilmente os componentes internos de qualquer hardware para entender mais detalhadamente o funcionamento interno.

- **Diagrama de implantação:** esses diagramas são mais úteis para engenheiros de sistemas e normalmente mostram desempenho, escalabilidade, capacidade de manutenção e portabilidade. Quando os componentes de hardware são exibidos em relação uns aos outros, fica mais fácil acompanhar toda a sua malha de hardware e garantir que todos os elementos sejam considerados em uma implantação.

- **Diagrama de objetos:** os diagramas de objetos mostram exemplos de estruturas de dados em um momento específico. Você pode usar um diagrama de classes para mostrar uma estrutura e, em seguida, usar diagramas de objetos como casos de teste para verificar a integridade do seu diagrama de classes. Ou você pode criar um diagrama de objetos para descobrir informações sobre elementos do modelo.

- **Diagrama de pacotes:** os diagramas de pacotes são usados para mostrar as dependências entre diferentes pacotes em um sistema. Um pacote, representado como uma pasta de arquivos, organiza elementos do modelo, como casos de uso ou classes, em grupos. Existem dois tipos especiais de dependências definidas entre pacotes: importação de pacote e fusão de pacotes.

- **Diagramas de perfil:** os diagramas de perfil também fazem parte do padrão UML e são usados principalmente quando você precisa adaptar a UML para um domínio específico. Adicionados recentemente à UML 2.0, os diagramas de perfil são únicos e raramente usados em qualquer especificação. Um diagrama de perfil é melhor compreendido como um mecanismo de extensibilidade para personalizar modelos UML para domínios e plataformas específicas.

Diagramas UML comportamentais

- **Diagramas de atividades:** os diagramas de atividades são usados como uma alternativa aos diagramas de máquina de estados. Os diagramas de atividades visualizam as etapas executadas em um caso de uso — as atividades podem ser sequenciais, ramificadas ou concorrentes. Esse tipo de diagrama UML é usado para mostrar o comportamento dinâmico de um sistema, mas também pode ser útil na modelagem de processos de negócios.

- **Diagrama de comunicação:** os diagramas de comunicação, que costumavam ser chamados de diagramas de colaboração, mostram como os objetos se relacionam entre si. Eles modelam a maneira como os objetos se associam e se conectam por meio de mensagens dentro do design arquitetônico de um sistema. Eles também podem mostrar cenários alternativos dentro de casos de uso ou operações que exigem a colaboração de diferentes objetos e interações.

- **Diagrama de visão geral de interação:** este diagrama fornece uma visão geral do fluxo de controle entre os nós de interação. Eles incluem nós iniciais, nós finais de fluxo, nós finais de atividade, nós de decisão, nós de fusão, nós

de bifurcação e nós de junção.

- **Diagrama de sequência:** um diagrama de sequência, às vezes chamado de diagrama de eventos ou cenário de eventos, mostrará a ordem em que os objetos interagem. Dessa forma, você pode representar visualmente cenários simples de tempo de execução.

- **Diagrama de estados:** os diagramas de estados, simplificando, representam estados e transições. Um estado refere-se às diferentes combinações de informações que um objeto pode conter, e esse diagrama UML pode visualizar todos os estados possíveis e a maneira como o objeto faz a transição de um estado para o próximo.

- **Diagrama de tempo:** frequentemente descrito como um diagrama de sequência invertido, um diagrama de tempo mostra como os objetos interagem entre si em um determinado período de tempo. Use esses diagramas para ver quanto tempo leva cada etapa de um processo e encontrar áreas de melhoria.

- **Diagrama de caso de uso:** os diagramas de caso de uso modelam como os usuários, exibidos como figuras de palito chamadas de "atores", interagem com o sistema. Esse tipo de diagrama UML deve ser uma visão geral de alto nível dos relacionamentos entre atores e sistemas, por isso pode ser uma ótima ferramenta para explicar seu sistema a um público não técnico.

Conceitos de modelagem especificados pela UML

O desenvolvimento de sistemas concentra-se em três modelos gerais de sistema diferentes:

- **Funcional:** são os diagramas de caso de uso, que descrevem a funcionalidade do sistema do ponto de vista do usuário.

- **Objeto:** são os diagramas de classes, que descrevem a estrutura do sistema em termos de objetos, atributos, associações e operações.

- **Dinâmico:** diagramas de interação, diagramas de máquina de estados e diagramas de atividades são usados para descrever o comportamento interno do sistema.

Esses modelos de sistema são visualizados por meio de dois tipos diferentes de diagramas: estruturais e comportamentais.

Conceitos orientados a objetos na UML

Os objetos na UML são entidades do mundo real que existem ao nosso redor. No desenvolvimento de software, os objetos podem ser usados para descrever, ou modelar, o sistema que está sendo criado em termos relevantes para o domínio. Os objetos também permitem a decomposição de sistemas complexos em componentes compreensíveis que permitem que uma parte seja construída por vez.

Aqui estão alguns conceitos fundamentais de um mundo orientado a objetos:

- **Objetos:** representam uma entidade e o bloco de construção básico
- **Classe:** planta de um objeto.
- **Abstração:** comportamento de uma entidade do mundo real
- **Encapsulamento:** mecanismo de agrupar os dados e ocultá-los do mundo externo
- **Herança:** mecanismo de criar novas classes a partir de uma existente
- **Polimorfismo:** define o mecanismo para existir em diferentes formas

Em equipes modernas, especialmente aquelas que praticam Agile, esses conceitos orientados a objetos costumam ser mais fáceis de comunicar quando os diagramas são tratados como “documentação principal” e atualizados juntamente com o código, em vez de serem criados uma única vez e deixados de lado.

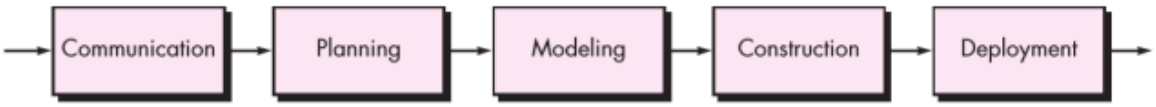
MÓDULO 3 — Fluxos e Modelos de Desenvolvimento

3.1 Fluxos de Softwares

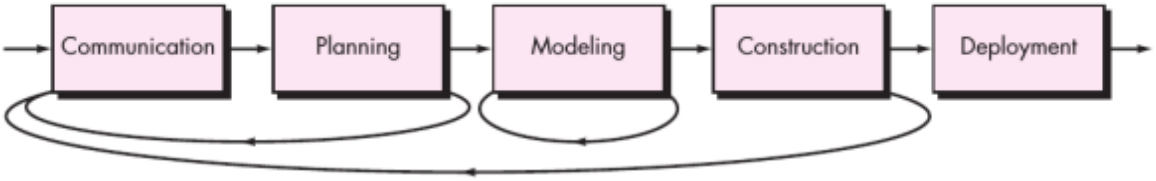
Explica como as atividades são organizadas, detalhando as ações e tarefas que ocorrem em cada uma, além de como elas se conectam em termos de sequência e tempo.

Dividimos eles em:

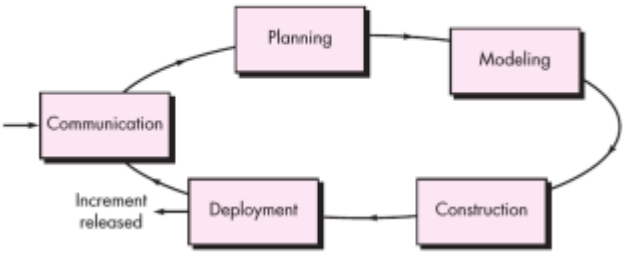
Linear: É um modelo em que as etapas são realizadas em uma sequência fixa e não permitem retornos. As etapas incluem o levantamento de requisitos, o projeto da estrutura do software, a codificação, a verificação e correção de erros, o lançamento para os usuários e, por fim, a manutenção, que envolve atualizações e correções após o lançamento. Cada etapa deve ser concluída antes de avançar para a próxima.



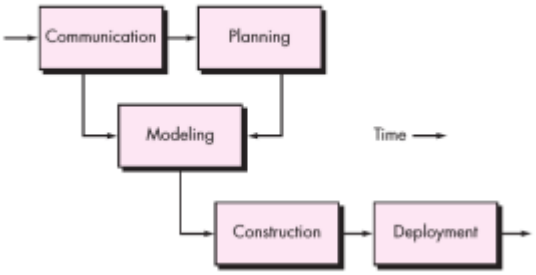
Interativo : É comum que tarefas sejam revisadas e repetidas conforme necessário, permitindo ajustes e melhorias antes de avançar para as próximas etapas. Além disso, pode haver a necessidade de voltar a etapas anteriores para corrigir problemas ou implementar novas ideias. Isso promove um desenvolvimento mais flexível e adaptável.



Evolucionário : É um modelo que funciona em ciclos. Primeiro, você planeja o que precisa, depois cria uma versão inicial do software e a testa para encontrar problemas. Em melhorias. Esse processo se repe às necessidades dos usuários ao l



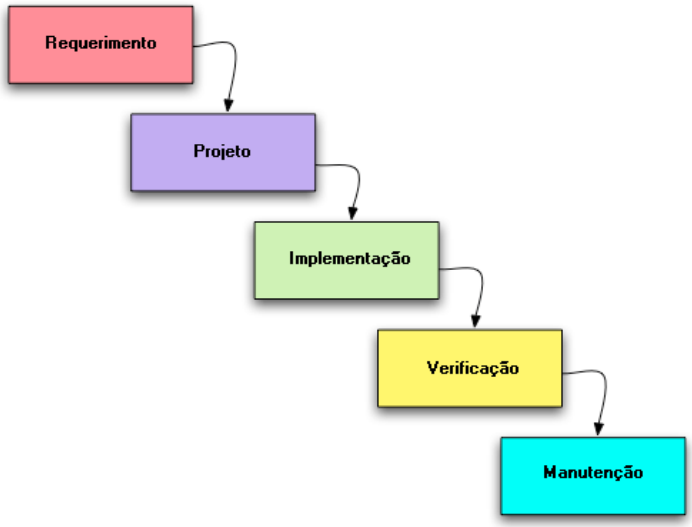
Paralelo : É um modelo em que várias atividades são realizadas ao mesmo tempo, em vez de seguirem uma sequência linear. Enquanto uma equipe trabalha na codificação, outra pode estar testando ou projetando novas funcionalidades. Isso permite que o desenvolvimento seja mais rápido e eficiente, pois diferentes partes do projeto avançam simultaneamente.



3.2 Os principais de Modelos de Processos de Software :

1. Modelo Cascata

Descrição Técnica: O modelo cascata é um processo sequencial onde cada fase do desenvolvimento de software deve ser concluída antes de passar para a próxima. As fases incluem: levantamento de requisitos, design, implementação, teste e manutenção. Este modelo é rígido e não permite revisões nas etapas já concluídas.



Principais características do Modelo Cascata

- **Estrutura linear e sequencial:** como uma cachoeira, o fluxo segue sempre para frente;
- **Documentação extensa:** cada fase gera documentos detalhados;
- **Controle rígido de escopo:** o que foi definido no início raramente muda;
- **Previsibilidade de custos e prazos:** orçamento e cronograma fixos desde o início;
- **Baixa participação do cliente:** envolvimento principalmente no começo e no fim.

Inicialmente, o modelo waterfall foi proposto como uma maneira de melhorar a previsibilidade dos projetos. Como ele é baseado em planejamento detalhado e documentação completa, oferece maior capacidade preditiva sobre tempo e orçamento para as empresas.

Desafios

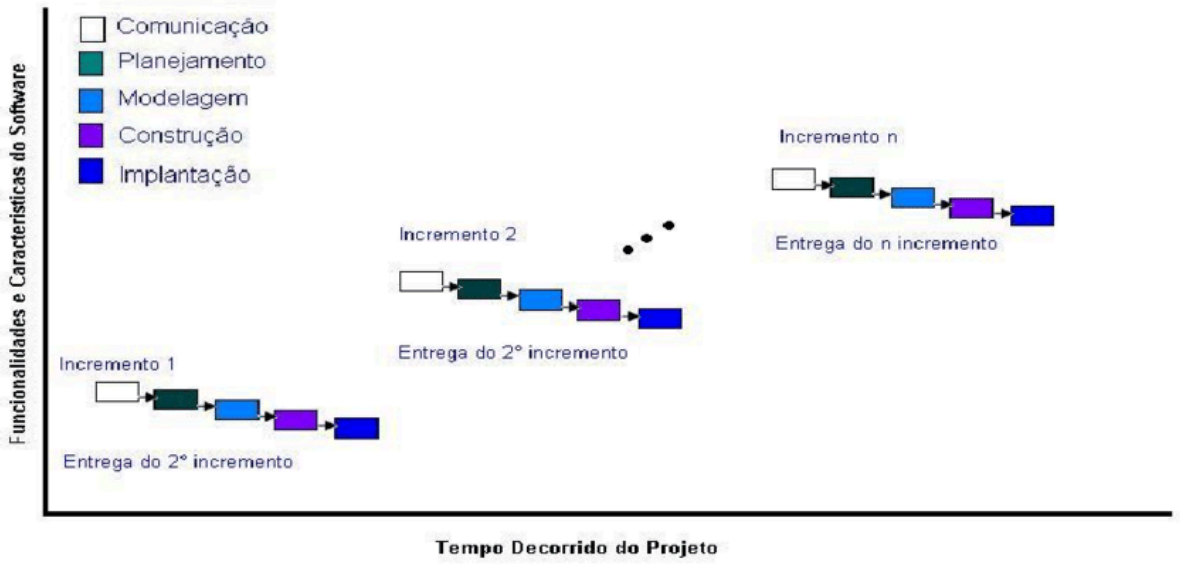
- O principal desafio do modelo cascata é a dificuldade de acomodar mudanças após o processo estar em execução. Em princípio, uma fase tem de ser concluída antes de se mover para a próxima fase.
- Maior dificuldade em responder à mudança de requisitos.

Onde o modelo cascata é mais empregado ?

- O modelo é apropriado em sistemas nos quais os requisitos são bem compreendidos e alterações do escopo do sistema estarão limitadas durante a fase de design.
- Poucos sistemas de negócios têm requisitos estáveis.

2. Desenvolvimento Incremental

Descrição Técnica: O desenvolvimento incremental é uma abordagem onde o software é desenvolvido em pequenas partes ou incrementos. Cada parte é planejada, projetada, codificada e testada individualmente. Isso permite que o software seja entregue mais rapidamente e que novas funcionalidades sejam adicionadas gradualmente.



3. Desenvolvimento Evolucionário

Descrição Técnica: O desenvolvimento evolucionário é um modelo que permite criar versões do software em ciclos. Cada versão é aprimorada com base no feedback dos usuários e nas necessidades que surgem ao longo do tempo. Isso torna o software mais adaptável e relevante.

Dentro do evolucionário temos os modelos de prototipação e o modelo espiral.

1. Modelo de Prototipação

Imagine que você quer construir um carro novo. Em vez de construir o carro todo de uma vez, você faz um modelo simples, um protótipo. Isso te ajuda a ver como o carro vai funcionar e quais mudanças são necessárias.

- **Como funciona**:
    - Primeiro, você cria uma versão básica do software (o protótipo).
    - Depois, mostra para os usuários e pergunta o que eles acham.
    - Com o feedback deles, você faz melhorias.
    - Esse processo se repete até que o software esteja do jeito certo.

- **Vantagens**:

- Ajuda a entender melhor o que os usuários precisam.
- Permite fazer mudanças antes que tudo esteja pronto.



2. Modelo Espiral

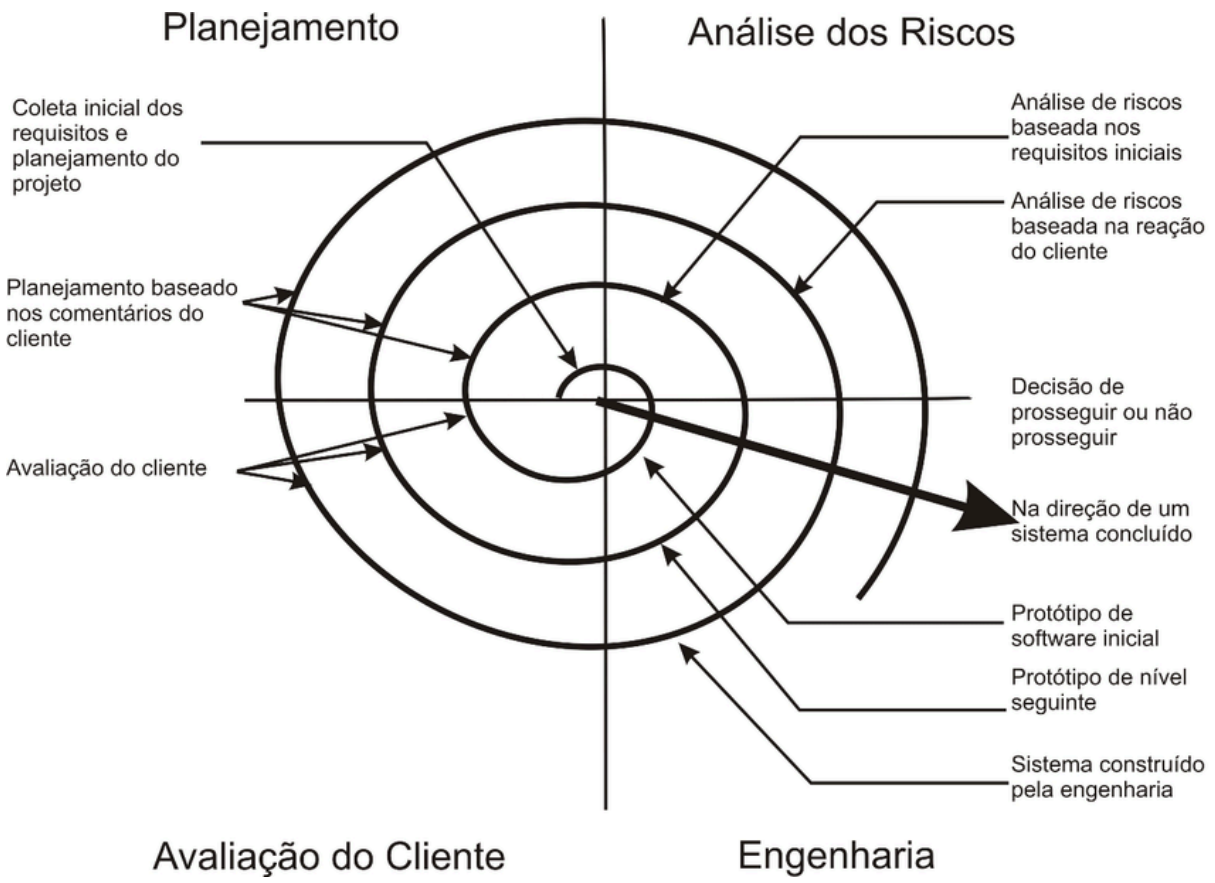
Agora, pense em um caracol. Ele vai construindo sua concha aos poucos, em espirais. O modelo espiral é parecido: você desenvolve o software em ciclos que vão se ampliando.

- **Como funciona**:
  - Cada ciclo começa com uma fase de planejamento e identificação de riscos.
  - Depois, você faz um pequeno desenvolvimento e cria uma versão do software.
  - Em seguida, testa essa versão e coleta feedback.
  - Finalmente, você ajusta e começa o próximo ciclo, sempre ampliando e melhorando.

- **Vantagens**:

- Permite lidar com riscos, já que você identifica problemas antes de avançar muito.

- É flexível e se adapta a mudanças ao longo do desenvolvimento.



4. Engenharia de Software Orientada a Reuso

A engenharia de software orientada a reuso é uma abordagem que enfatiza a reutilização de componentes de software já existentes. Em vez de criar tudo do zero, desenvolvedores usam módulos ou bibliotecas que já foram feitos anteriormente, economizando tempo e esforço.



3.3 RUP (Rational Unified Process)

É um processo de desenvolvimento de software que se baseia em uma abordagem iterativa e incremental. Ele foi desenvolvido pela Rational Software e é amplamente utilizado para gerenciar o desenvolvimento de sistemas complexos. O RUP é flexível e adaptável, permitindo que as equipes personalizem o processo de acordo com as necessidades do projeto.

Fases do RUP

O RUP é dividido em quatro fases principais, cada uma com suas atividades e objetivos específicos:

1. **Iniciação (Inception)**:
    - **Objetivo**: Definir o escopo do projeto e identificar os principais requisitos e riscos.
    - **Atividades**:
        - Identificação de stakeholders.
        - Levantamento de requisitos iniciais.
        - Análise de viabilidade.
        - Criação de um esboço do planejamento do projeto.
    - **Saída**: Documento de visão e um entendimento básico do que o projeto deve alcançar.

2. **Elaboração (Elaboration)**:

    - **Objetivo**: Refinar os requisitos e a arquitetura do sistema, reduzindo os riscos identificados.

    - **Atividades**:
        - Desenvolvimento de uma arquitetura inicial.
        - Criação de um protótipo e testes iniciais.
        - Detalhamento dos requisitos funcionais e não funcionais.
        - Planejamento mais detalhado.

    - **Saída**: Um modelo de arquitetura sólida e um plano de desenvolvimento detalhado.

3. **Construção (Construction)**:

    - **Objetivo**: Desenvolver o software conforme os requisitos e arquitetura definidos nas fases anteriores.

    - **Atividades**:
        - Codificação e testes das funcionalidades.
        - Integração de componentes.
        - Testes contínuos e validação do software.

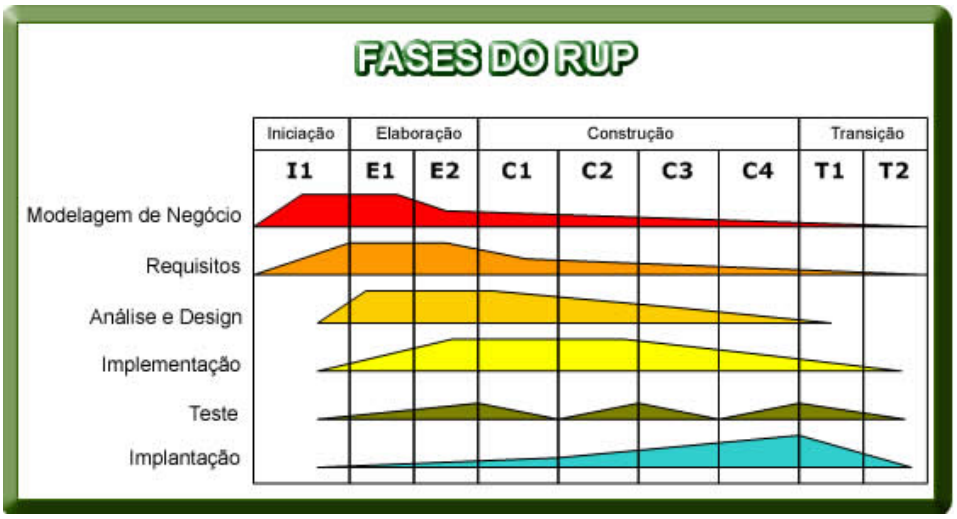
    - **Saída**: O produto de software em sua forma quase final, pronto para testes e ajustes.

4. **Transição (Transition)**:

    - **Objetivo**: Preparar o software para ser entregue aos usuários finais.

    - **Atividades**:
        - Testes finais e correções de bugs.
        - Treinamento dos usuários.
        - Implantação do software no ambiente de produção.
        - Coleta de feedback dos usuários.

    - **Saída**: O produto final entregue e um plano de suporte pós-implantação.


FASES DO RUP
Iniciação
Elaboração
Construção
Transição
I1
E1
E2
C1
C2
C3
C4
T1
T2
Modelagem de Negócio
Requisitos
Análise e Design
Implementação
Teste
Implantação

MÓDULO 4 — Métodos Ágeis e Gerenciamento

4.1 Metodologia Ágil

A Metodologia Ágil é uma abordagem de desenvolvimento de software baseada em ciclos curtos, colaborativos e interativos. Em vez de planejar e entregar o projeto inteiro apenas no final, a equipe constrói e entrega partes funcionais do produto continuamente. Isso permite adaptação rápida a mudanças, mitigação de riscos e absorção constante do feedback do cliente.

Vantagens de usar a metodologia Ágil em seu fluxo de trabalho

A flexibilidade da metodologia Agile permite que as empresas cresçam e respondam às mudanças mais rapidamente. Abaixo estão alguns dos benefícios de implementar um fluxo de trabalho Ágil:

- **Dá mais flexibilidade**
  Você tem mais flexibilidade com um fluxo de trabalho ágil, como já falamos. Ao contrário de muitos fluxos de trabalho tradicionais, não há uma estrutura rígida, já que o workload é sempre revisto a cada sprint. Ao final de cada sprint, você pode rever seu processo e fazer mudanças para melhorá-lo.

- **Melhora a colaboração**
  As equipes ágeis são, por natureza, colaborativas. Por exemplo, a equipe sempre se reúne ao final de um sprint para rever sucessos e fracassos e decidir quais itens priorizar a partir do product backlog (mais sobre isso mais adiante). Então, eles se esforçam como equipe para alcançar a meta do sprint.

- **Oferece mais valor ao cliente**
  Pode parecer sutil ou indireto, mas ao usar a metodologia Ágil, você se concentra na melhoria contínua, já que o formato das reuniões e fluxo de trabalho funciona em ciclos. Isso, consequentemente, potencializa um processo melhor para desenvolvimento de produtos e serviços. Além disso, quando seus processos correm sem problemas, suas equipes de vendas e atendimento ao cliente podem se concentrar em fornecer aos clientes a melhor experiência possível sem se preocupar com gargalos internos.

- **Prioriza tarefas**
  O product backlog (backlog de produto) é uma parte fundamental do fluxo de trabalho Ágil. Ao início de cada sprint, a equipe revisa o backlog de produto e dá prioridade às tarefas mais importantes, assegurando que seu fluxo de trabalho seja o mais tranquilo e perfeito possível.

Ágil vs. Tradicional

O fluxo tradicional (frequentemente chamado de modelo Cascata ou Waterfall) é estruturado de forma linear e rígida, enquanto o Ágil é construído para ser flexível.

Característica	Fluxo Tradicional (Cascata)	Fluxo Ágil
Estrutura	Fases sequenciais dependentes (Planejar > Projetar > Construir > Testar > Entregar).	Ciclos iterativos (Sprints) onde planejamento, construção e testes acontecem simultaneamente.
Flexibilidade	Baixa. Mudanças no escopo após o início são difíceis, lentas e custosas.	Alta. O escopo é reavaliado a cada ciclo para se adaptar a novas necessidades.
Entrega de Valor	O cliente só vê o produto funcionando no final do cronograma.	O cliente recebe partes funcionais (incrementos) frequentemente.
Foco Principal	Seguir estritamente o plano inicial e manter uma documentação extensa.	Colaboração constante, software em funcionamento e resposta rápida à mudança.

4.2 Frameworks e práticas Ágeis

Embora o termo "Ágil" seja o guarda-chuva, existem vários métodos (frameworks) para aplicá-lo na prática:

- **Scrum:** O mais popular. Divide o trabalho em ciclos de tempo fixo (geralmente de 2 a 4 semanas) chamados *Sprints*, com papéis bem definidos e reuniões regulares de alinhamento.

- **Scrumban:** Um híbrido que combina a estrutura do Scrum com a fluidez do

Kanban. Geralmente adota as cerimônias e papéis do Scrum, mas remove a rigidez dos Sprints em favor do fluxo contínuo e dos limites de WIP do Kanban. Muito usado por equipes de suporte ou manutenção.

- **Kanban:** Focado na visualização contínua do fluxo de trabalho através de um quadro (board) e na limitação do trabalho em progresso (WIP) para evitar gargalos e otimizar a velocidade de entrega.

- **Extreme Programming (XP):** Muito focado na qualidade técnica e na engenharia de código, enfatizando práticas como testes contínuos, programação em pares (Pair Programming) e refatoração constante.

- **Lean Software Development:** Originado na manufatura, concentra-se na eliminação implacável de desperdícios (qualquer coisa que não gere valor ao cliente), tomada de decisões o mais tarde possível e entrega o mais rápido possível.
- **Test Driven Development (TDD):** (Desenvolvimento Orientado a Testes) Não é um framework de gestão, mas uma prática técnica. O programador escreve o teste automatizado antes de escrever o código da funcionalidade. O ciclo é: escrever um teste que falha (Red), escrever o código mínimo para o teste passar (Green) e melhorar o código (Refactor).

- **Feature Driven Development (FDD):** Metodologia orientada por modelos e focada no design e construção por funcionalidades (features). O processo começa com a modelagem do domínio do negócio, seguida pela criação de uma lista de funcionalidades e, então, o planejamento e construção iterativa de cada uma delas.

- **Adaptive Software Development (ASD):** Baseia-se na ideia de que o desenvolvimento é complexo e imprevisível. Substitui o ciclo tradicional de "Planejar-Construir-Testar" pelo ciclo Especular (Speculate), Colaborar (Collaborate) e Aprender (Learn), focando em adaptação constante em vez de otimização rígida.

- **Scaled Agile Framework (SAFe):** Um framework robusto criado para escalar o Ágil em grandes corporações. Ele pega práticas do Scrum, Kanban, Lean e XP e fornece uma estrutura para aplicá-las em múltiplos times simultaneamente, alinhando a estratégia de negócios da alta gestão com a execução técnica de centenas de desenvolvedores.

4.3 Scrum e seus papéis

O Scrum define três responsabilidades claras e exclusivas para garantir que o ciclo ágil funcione perfeitamente, sem a necessidade de um "gerente de projeto" tradicional.

1. Product Owner (PO)

É a voz do cliente, o dono da visão de negócios e quem entende o que o mercado precisa.

- **Responsabilidade:** Maximizar o valor do produto entregue pela equipe.
- **Ações:** Ele define os requisitos, escreve o que precisa ser feito e prioriza a lista de tarefas (*Product Backlog*). Ele diz *o que* precisa ser construído e *por que*, mas não interfere no *como*.

2. Scrum Master (SM)

É o facilitador e o guardião da cultura ágil. Ele atua como um líder servidor para a equipe.

- **Responsabilidade:** Garantir que o framework Scrum seja compreendido e aplicado corretamente por todos.
- **Ações:** Ele protege a equipe de interrupções externas, remove obstáculos (impedimentos) que estejam travando o trabalho dos desenvolvedores e facilita as cerimônias (como as reuniões diárias e retrospectivas).

3. Desenvolvedores (Developers)

É a equipe técnica multidisciplinar (programadores, designers, testadores) responsável por colocar a mão na massa.

- **Responsabilidade:** Criar o incremento funcional do produto a cada ciclo com alta qualidade.

Ações: A equipe é auto-organizada. Eles estimam o esforço das tarefas, puxam o trabalho priorizado pelo PO e decidem internamente *como* tecnicamente transformar aquele requisito em um software funcionando até o final da Sprint.

4.4 PMBOK

PMBOK significa Project Management Body of Knowledge (**Conjunto de Conhecimentos de Gerenciamento de Projetos** na tradução). Trata-se de um guia que descreve práticas, diretrizes e até terminologias padronizadas no setor de gestão de projetos.

Desse modo, gestores de diferentes setores podem balizar o gerenciamento de processos seguindo as mesmas diretrizes, atingindo assim resultados mais previsíveis, bem como auxiliando na prevenção de falhas. Vale destacar que o Project Management Body of Knowledge **não é uma metodologia**, mas uma estrutura que organiza conhecimentos e orienta a aplicação de práticas de gerenciamento de projetos, podendo ser adaptado à realidade de cada organização. É justamente por isso que falamos que o PMBOK é como a base de metodologias mais fundamentadas, como é o caso do *Adaptive Project Framework* (APF).

O Guia PMBOK (8ª Edição)

O PMBOK (Project Management Body of Knowledge) é um guia de boas práticas para gerenciamento de projetos adaptável a qualquer metodologia.

Princípio do PMBOK (8ª ed.)	O que enfatiza
Adote uma visão holística	Incentiva o pensamento sistêmico e a consciência de como as decisões afetam o ambiente organizacional amplo.
Foque no valor	Reforça que o sucesso é medido por resultados que importam para os stakeholders, não apenas por conclusão de tarefas.
Qualidade de incorporação	Promove a integração proativa da qualidade nos processos e entregas ao longo do ciclo do projeto.
Seja um líder responsável	Combina gestão responsável e liderança para apoiar decisões éticas e boa governança.
Integrar a Sustentabilidade	Enfatiza considerações ambientais, econômicas e sociais como parte da gestão responsável.

Princípio do PMBOK (8ª ed.)	O que enfatiza
Construir uma cultura de empoderamento	Promove colaboração, confiança, segurança psicológica e forte desempenho da equipe.

MÓDULO 5 — Orientação a Objetos, Arquitetura e Design

5.1 POO: abstração, encapsulamento, herança e polimorfismo

O desenvolvimento de software é extremamente amplo. Nesse mercado, existem diversas linguagens de programação, que seguem diferentes paradigmas. Um desses paradigmas é a **Orientação a Objetos**, que atualmente é o mais difundido entre todos. Isso acontece porque se trata de um padrão que tem evoluído muito, principalmente em questões voltadas para segurança e reaproveitamento de código, o que é muito importante no desenvolvimento de qualquer aplicação moderna.

A **Programação Orientada a Objetos (POO)** diz respeito a um padrão de desenvolvimento que é seguido por muitas linguagens, como C# e Java.VAmos entender as diferenças entre a POO e a **Programação Estruturada**, que era muito utilizada há alguns anos, principalmente com a linguagem C.

Programação Estruturada vs Programação Orientada a Objetos

Na **Figura 1** vemos uma comparação muito clara entre a programação estruturada e a **programação orientada a objetos** no que diz respeito aos dados. Repare que, no paradigma estruturado, temos procedimentos (ou funções) que são aplicados globalmente em nossa aplicação. No caso da orientação a objetos, temos métodos que são aplicados aos dados de cada objeto. Essencialmente, os procedimentos e métodos são iguais, sendo diferenciados apenas pelo seu escopo.



Figura 1. Estruturada x Orientação a Objetos

A linguagem C é a principal representante da programação estruturada. Trata-se de

uma linguagem considerada de baixo nível, que atualmente não é utilizada para projetos muito grandes. A sua principal utilização, devido ao baixo nível, é em programação para sistemas embarcados ou outros em que o conhecimento do hardware se faz necessário para um bom programa.

Essa colocação nos traz a um detalhe importante: a programação estruturada, quando bem feita, possui um desempenho superior ao que vemos na programação orientada a objetos. Isso ocorre pelo fato de ser um paradigma sequencial, em que cada linha de código é executada após a outra, sem muitos desvios, como vemos na **POO**. Além disso, o paradigma estruturado costuma permitir mais liberdades com o hardware, o que acaba auxiliando na questão de desempenho.

Entretanto, a programação orientada a objetos traz outros pontos que acabam sendo mais interessantes no contexto de aplicações modernas. Como o desempenho das aplicações não é uma das grandes preocupações na maioria das aplicações (devido ao poder de processamento dos computadores atuais), a programação orientada a objetos se tornou muito difundida. Essa difusão se dá muito pela questão da reutilização de código e pela capacidade de representação do sistema muito mais perto do que veríamos no mundo real.

Veremos em detalhes esses e outros pontos que dizem respeito à programação **orientada a objetos**. Como desenvolvedores, é nossa missão entender quais são as vantagens e desvantagens de cada um dos paradigmas de programação e escolhermos o melhor para nossa aplicação. A escolha da linguagem também deve estar presente nessa escolha.

Os 4 pilares da Programação Orientada a Objetos

Para entendermos exatamente do que se trata a orientação a objetos, vamos entender quais são os requerimentos de uma linguagem para ser considerada nesse paradigma. Para isso, a linguagem precisa atender a quatro tópicos bastante importantes:

Abstração

A abstração consiste em um dos pontos mais importantes dentro de qualquer linguagem **Orientada a Objetos**. Como estamos lidando com uma representação de um objeto real (o que dá nome ao paradigma), temos que imaginar o que esse objeto irá realizar dentro de nosso sistema. São três pontos que devem ser levados em consideração nessa abstração.

O primeiro ponto é darmos uma ***identidade*** ao objeto que iremos criar. Essa identidade deve ser única dentro do sistema para que não haja conflito. Na maior parte das linguagens, há o conceito de pacotes (ou namespaces). Nessas

linguagens, a identidade do objeto não pode ser repetida dentro do pacote, e não necessariamente no sistema inteiro. Nesses casos, a identidade real de cada objeto se dá por ..

A segunda parte diz respeito a características do objeto. Como sabemos, no mundo real qualquer objeto possui elementos que o definem. Dentro da programação orientada a objetos, essas características são nomeadas ***propriedades***. Por exemplo, as propriedades de um objeto “Cachorro” poderiam ser “Tamanho”, “Raça” e “Idade”.

Por fim, a terceira parte é definirmos as ações que o objeto irá executar. Essas ações, ou eventos, são chamados ***métodos***. Esses métodos podem ser extremamente variáveis, desde “Acender()” em um objeto lâmpada até “Latir()” em um objeto cachorro.

Encapsulamento

O *encapsulamento* é uma das principais técnicas que define a programação orientada a objetos. Se trata de um dos elementos que adicionam segurança à aplicação em uma programação orientada a objetos pelo fato de esconder as propriedades, criando uma espécie de caixa preta.

A maior parte das linguagens orientadas a objetos implementam o encapsulamento baseado em propriedades privadas, ligadas a métodos especiais chamados *getters* e *setters*, que irão retornar e setar o valor da propriedade, respectivamente. Essa atitude evita o acesso direto a propriedade do objeto, adicionando uma outra camada de segurança à aplicação.

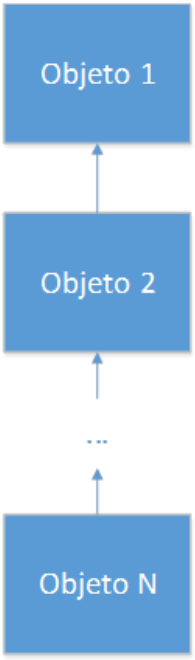
Para fazermos um paralelo com o que vemos no mundo real, temos o encapsulamento em outros elementos. Por exemplo, quando clicamos no botão ligar da televisão, não sabemos o que está acontecendo internamente. Podemos então dizer que os métodos que ligam a televisão estão encapsulados.

Herança

O reuso de código é uma das grandes vantagens da programação orientada a objetos. Muito disso se dá por uma questão que é conhecida como *herança*. Essa característica otimiza a produção da aplicação em tempo e linhas de código.

Para entendermos essa característica, vamos imaginar uma família: a criança, por exemplo, está herdando características de seus pais. Os pais, por sua vez, herdam algo dos avós, o que faz com que a criança também o faça, e assim sucessivamente. Na orientação a objetos, a questão é exatamente assim, como mostra a **Figura 2**. O objeto abaixo na hierarquia irá herdar características de todos os objetos acima dele, seus “ancestrais”. A herança a partir das características do

objeto mais acima é considerada herança direta, enquanto as demais são consideradas heranças indiretas. Por exemplo, na família, a criança herda diretamente do pai e indiretamente do avô e do bisavô.



A questão da herança varia bastante de linguagem para linguagem. Em algumas delas, como C++, há a questão da herança múltipla. Isso, essencialmente, significa que o objeto pode herdar características de vários “ancestrais” ao mesmo tempo diretamente. Em outras palavras, cada objeto pode possuir quantos pais for necessário. Devido a problemas, essa prática não foi difundida em linguagens mais modernas, que utilizam outras artimanhas para criar uma espécie de herança múltipla.

Outras linguagens orientadas a objetos, como C#, trazem um objeto base para todos os demais. A classe *object* fornece características para todos os objetos em C#, sejam criados pelo usuário ou não.

Polimorfismo

Outro ponto essencial na programação orientada a objetos é o chamado polimorfismo. Na natureza, vemos animais que são capazes de alterar sua forma conforme a necessidade, e é dessa ideia que vem o polimorfismo na orientação a objetos. Como sabemos, os objetos filhos herdam as características e ações de seus “ancestrais”. Entretanto, em alguns casos, é necessário que as ações para um mesmo método seja diferente. Em outras palavras, o *polimorfismo* consiste na alteração do funcionamento interno de um método herdado de um objeto pai.

Como um exemplo, temos um objeto genérico “Eletrodoméstico”. Esse objeto possui um método, ou ação, “Ligar()”. Temos dois objetos, “Televisão” e “Geladeira”, que não irão ser ligados da mesma forma. Assim, precisamos, para cada uma das classes filhas, reescrever o método “Ligar()”.

Com relação ao polimorfismo, valem algumas observações. Como se trata de um assunto que está intimamente conectado à herança, entender os dois juntamente é uma boa ideia. Outro ponto é o fato de que as linguagens de programação implementam o polimorfismo de maneiras diferentes. O C#, por exemplo, faz uso de método virtuais (com a palavra-chave *virtual*) que podem ser implementados (com a palavra-chave *override*) nas classes filhas. Já em Java, apenas o atributo “@Override” é necessário.

Esses quatro pilares são essenciais no entendimento de qualquer linguagem orientada a objetos e da orientação a objetos como um todo. Cada linguagem irá implementar esses pilares de uma forma, mas essencialmente é a mesma coisa. Apenas a questão da herança, como comentado, que pode trazer variações mais bruscas, como a presença de herança múltipla. Além disso, o encapsulamento também é feito de maneiras distintas nas diversas linguagens, embora os *getters* e *setters* sejam praticamente onipresentes.

Principais vantagens da POO

A programação orientada a objetos traz uma ideia muito interessante: a representação de cada elemento em termos de um objeto, ou classe. Esse tipo de representação procura aproximar o sistema que está sendo criado ao que é observado no mundo real, e um objeto contém características e ações, assim como vemos na realidade. Esse tipo de representação traz algumas vantagens muito interessantes para os desenvolvedores e também para o usuário da aplicação. Veremos algumas delas a seguir.

A reutilização de código é um dos principais requisitos no desenvolvimento de software atual. Com a complexidade dos sistemas cada vez maior, o tempo de desenvolvimento iria aumentar exponencialmente caso não fosse possível a reutilização. A orientação a objetos permite que haja uma reutilização do código criado, diminuindo o tempo de desenvolvimento, bem como o número de linhas de código. Isso é possível devido ao fato de que as linguagens de programação orientada a objetos trazem representações muito claras de cada um dos elementos, e esses elementos normalmente não são interdependentes. Essa independência entre as partes do software é o que permite que esse código seja reutilizado em outros sistemas no futuro.

Outra grande vantagem que o desenvolvimento orientado a objetos traz diz respeito à leitura e manutenção de código. Como a representação do sistema se aproxima

muito do que vemos na vida real, o entendimento do sistema como um todo e de cada parte individualmente fica muito mais simples. Isso permite que a equipe de desenvolvimento não fique dependente de uma pessoa apenas, como acontecia com frequência em linguagens estruturadas como o C, por exemplo.

A criação de bibliotecas é outro ponto que é muito mais simples com a orientação a objetos. No caso das linguagens estruturadas, como o C, temos que as bibliotecas são coleções de procedimentos (ou funções) que podem ser reutilizadas. No caso da **POO**, entretanto, as bibliotecas trazem representações de classes, que são muito mais claras para permitirem a reutilização.

Entretanto, nem tudo é perfeição na programação orientada a objetos. A execução de uma aplicação orientada a objetos é mais lenta do que o que vemos na programação estruturada, por exemplo. Isso acontece devido à complexidade do modelo, que traz representações na forma de classes. Essas representações irão fazer com que a execução do programa tenha muitos desvios, diferente da execução sequencial da programação estruturada. Esse é o grande motivo por trás da preferência pela linguagem C em hardware limitado, como sistemas embarcados. Também é o motivo pelo qual a programação para sistemas móveis como o Google Android, embora em Java (linguagem orientada a objetos), seja feita o menos orientada a objetos possível.

No momento atual em que estamos, tecnologicamente essa execução mais lenta não é sentida. Isso significa que, em termos de desenvolvimento de sistemas modernos, a programação orientada a objetos é a mais recomendada devido às vantagens que foram apresentadas. Essas vantagens são derivadas do modelo de programação, que busca uma representação baseada no que vemos no mundo real.

Exemplos de Linguagens Orientadas a Objetos

Há uma grande quantidade de linguagens de programação orientada a objetos no mercado atualmente. Nesse artigo, iremos apresentar 3 das mais utilizadas no momento: Java, C# e C++. Cada uma delas possui uma abordagem diferente do problema que as torna muito boas para alguns tipos de aplicações e não tão boas para outros.

5.2 SOLID

O SOLID é um acrônimo que representa cinco princípios fundamentais da programação orientada a objetos e do design de software, formulados por Robert C. Martin (Uncle Bob). O objetivo desses princípios é facilitar a criação de softwares mais compreensíveis, flexíveis, testáveis e fáceis de manter, evitando o "código espaguete" e o acoplamento excessivo.

Abaixo está o detalhamento de cada um dos cinco princípios.

1. S - Single Responsibility Principle (Princípio da Responsabilidade Única)

Definição: Uma classe deve ter um, e apenas um, motivo para mudar. Isso significa que ela deve ter uma única responsabilidade ou escopo de atuação dentro do sistema.

O Problema: Quando uma classe assume múltiplas funções (conhecida como *God Class* ou Classe Deus), qualquer alteração em uma regra de negócio pode quebrar outras funcionalidades não relacionadas.

Exemplo Prático: Imagine uma classe GerenciadorDePedidos. Se ela calcula o total do pedido, aplica descontos, gera um arquivo PDF do recibo e salva os dados no banco de dados, ela está violando o SRP. A Solução: Dividir as responsabilidades. Você teria:

- CalculadoraDePedidos: Apenas lida com a matemática e regras de negócio.
- GeradorDeReciboPdf: Lida exclusivamente com a formatação e geração do arquivo.
- PedidoRepository: Lida apenas com a comunicação com o banco de dados.

2. O - Open/Closed Principle (Princípio do Aberto/Fechado)

Definição: Entidades de software (classes, módulos, funções) devem estar abertas para extensão, mas fechadas para modificação.

O Problema: Toda vez que um novo requisito chega, você precisa alterar o código fonte de uma classe existente, correndo o risco de introduzir bugs em funcionalidades que já estavam prontas e testadas.

Exemplo Prático: Uma classe ProcessadorDePagamento que tem um bloco switch/case gigante ou múltiplos if/else para processar pagamentos: se for Cartão de Crédito faz uma coisa, se for Boleto faz outra. Se a empresa decidir aceitar Pix, você terá que modificar essa classe. A Solução: Usar polimorfismo. Crie uma interface

MetodoDePagamento com um método processar(). Crie classes separadas (PagamentoCartao, PagamentoBoleto, PagamentoPix) que implementam essa interface. O seu ProcessadorDePagamento apenas chama o método processar() da interface. Para adicionar o Pix, você cria um novo arquivo (extensão), sem tocar no processador existente (fechado para modificação).

3. L - Liskov Substitution Principle (Princípio da Substituição de Liskov)

Definição: Objetos de uma classe derivada (subclasse) devem poder substituir objetos da classe base (superclasse) sem quebrar a aplicação ou alterar o comportamento esperado do programa.

O Problema: Herança mal utilizada. Uma classe filha herda de uma classe pai, mas altera drasticamente o comportamento de um método ou lança exceções inesperadas para métodos que não faz sentido ela implementar.

Exemplo Prático: A classe Pinguim herda da classe Ave. A classe Ave tem um método voar(). Como pinguins não voam, o método voar() em Pinguim lança um erro NotImplementedException. Se um sistema recebe uma lista de aves e tenta fazer todas voarem, o sistema vai quebrar ao chegar no pinguim. A Solução: Repensar a abstração. Pode-se criar uma classe base Ave genérica, e interfaces ou subclasses menores como AveQueVoa e AveQueNada. O Pinguim herdaria apenas as características corretas, garantindo que qualquer AveQueVoa passada para o sistema realmente consiga voar.

4. I - Interface Segregation Principle (Princípio da Segregação de Interfaces)

Definição: Clientes não devem ser forçados a depender de interfaces que não utilizam. É melhor ter várias interfaces específicas do que uma única interface genérica ("gorda").

O Problema: Obrigar uma classe a implementar métodos que ela não precisa, deixando-os em branco ou lançando exceções, apenas para cumprir o contrato de uma interface mal desenhada.

Exemplo Prático: Uma interface genérica IMaquinaTrabalho exige os métodos imprimir(), escanear() e enviarFax(). Uma classe ImpressoraBasica implementa essa interface, mas como ela não escaneia nem envia fax, esses métodos ficam vazios. A Solução: Segregar a interface. Crie interfaces menores: IImpressora, IEscaner e IFax. A ImpressoraBasica implementará apenas IImpressora. Já uma classe ImpressoraMultifuncional pode implementar as três interfaces simultaneamente.

5. D - Dependency Inversion Principle (Princípio da Inversão de Dependência)

Definição: Módulos de alto nível não devem depender de módulos de baixo nível. Ambos devem depender de abstrações (interfaces). Além disso, abstrações não devem depender de detalhes; detalhes devem depender de abstrações.

O Problema: Alto acoplamento. Uma classe de regra de negócio instancia diretamente ferramentas de infraestrutura (bancos de dados, APIs de email), tornando-se dependente de uma tecnologia específica e quase impossível de testar isoladamente.

Exemplo Prático: Uma classe ServicoDeUsuario dá um new MySQLConnection() dentro do seu próprio código para salvar o usuário. Se a empresa migrar para PostgreSQL, a classe ServicoDeUsuario inteira precisará ser reescrita. A Solução: Injeção de dependência. A classe ServicoDeUsuario deve depender de uma interface IUsuarioRepository. A implementação concreta (como MySQLUsuarioRepository) é injetada (passada via construtor, por exemplo) para dentro do serviço de usuário. O serviço passa a desconhecer qual banco está sendo usado, focando apenas nas regras de negócio.

A adoção conjunta desses cinco princípios transforma o desenvolvimento de software. Embora exijam mais planejamento inicial e frequentemente aumentem o número de arquivos e pastas no projeto, o ganho a médio e longo prazo em escalabilidade, facilidade de manutenção e segurança nas refatorações compensa totalmente o esforço arquitetural.

5.3 Arquitetura em camadas

A arquitetura em camadas é um estilo arquitetural muito comum em aplicações de software, especialmente em sistemas corporativos e aplicações web.

Ela organiza o sistema em camadas lógicas, onde cada uma tem responsabilidades bem definidas. Essas camadas podem ser distribuídas de diferentes formas em camadas físicas (servidores, processos, containers etc.).

Camadas Lógicas Típicas

- Apresentação (UI/Web/API): Responsável por interações com o usuário ou com outras aplicações.

- Aplicação / Serviço: Orquestra a lógica entre apresentação e domínio.

- Domínio / Negócio: Contém regras de negócio puras, entidades, serviços de domínio etc.

- Infraestrutura / Acesso a dados: Comunicação com banco de dados, serviços externos, armazenamento etc.

Configurações Físicas Possíveis

As camadas lógicas podem ser organizadas em diferentes configurações físicas:

1. Camadas lógicas em um único processo (monolito)

Todas as camadas estão no mesmo servidor/aplicação. Por isso, é simples de implantar, testar e depurar, mas é pouco escalável. Pode virar um monólito difícil de manter.

2. Separação em múltiplos processos (ex: API + banco separado)

UI em um processo, lógica de aplicação em outro, banco em outro. Esse formato melhora a escalabilidade e isolamento, mas aumenta a complexidade de deploy e comunicação.

3. Distribuição por serviços (microserviços)

Cada camada lógica ou funcionalidade é um serviço separado (ex: front-end, gateway, serviço de pagamentos, serviço de produtos). Esse formato traz alta escalabilidade e independência de deploy. Porém. é muito maior a complexidade de orquestração, observabilidade e versionamento.

4. Camadas combinadas (ex: front + lógica juntos)

Em algumas soluções (ex: serverless, JAMstack), a apresentação e lógica podem estar no mesmo lugar. Por isso, esse modelo é rápido de desenvolver e hospedar. Mas ele pode limitar separação de responsabilidades e reutilização.

Vantagens da arquitetura em camadas

Com o uso dessa arquitetura, há uma separação de responsabilidades, pois facilita o entendimento, manutenção e evolução do sistema. Além disso, nesse estilo cada camada tem um papel específico (ex: apresentação, lógica de negócio, acesso a dados).

Também é possível realizar a reutilização de código, na qual camadas como a de acesso a dados podem ser reutilizadas por diferentes partes do sistema. Esse estilo arquitetural pode melhorar a testabilidade, isso porque os componentes podem ser testados isoladamente com mocks e stubs das outras camadas. Há também

melhoria na facilidade de manutenção, pois mudanças em uma camada, se bem isoladas, não afetam diretamente as outras.

E, por fim, a escalabilidade organizacional é uma vantagem, isso porque as equipes podem trabalhar de forma paralela em diferentes camadas (ex: front-end e back-end).

Desvantagens da arquitetura em camadas

Uma desvantagem é que pode haver overhead de chamadas entre camadas, especialmente se muitas camadas forem distribuídas fisicamente. Isso comprometeria a performance sistêmica. Além disso, esse estilo arquitetural possui certa rigidez, pois às vezes força uma passagem artificial de dados entre camadas, mesmo quando não é necessário.

Podem ocorrer dificuldades em mudanças transversais, pois as alterações que atravessam várias camadas (ex: rastreamento, auditoria) podem ser mais complexas. Esse modelo de Arquitetura exige planejamento cuidadoso e se não for bem pensado pode atrasar entregas iniciais em projetos com prazos curtos.

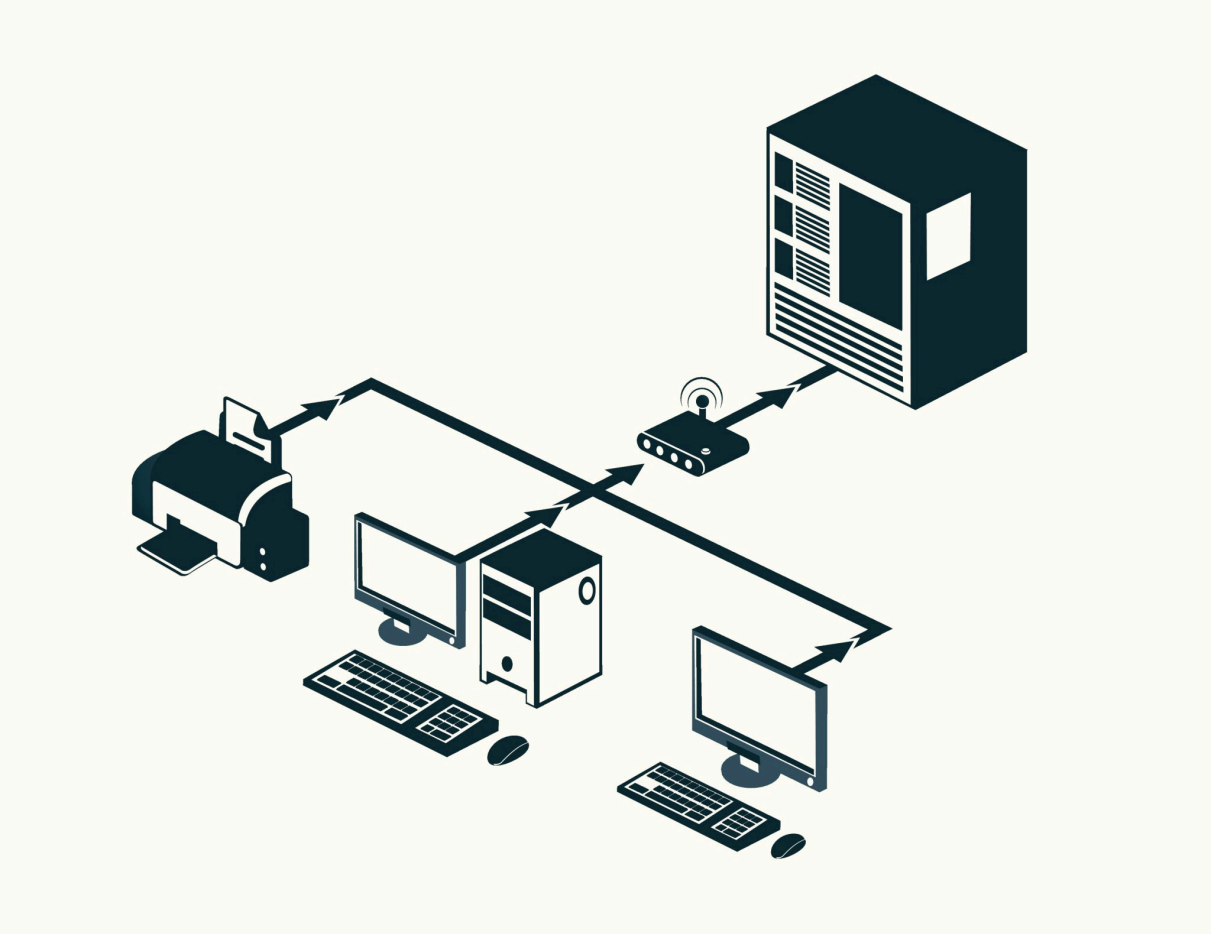
5.3 Cliente-servidor e microsserviços

Tanto "Cliente-Servidor" quanto "Microsserviços" são estilos arquiteturais de software. O primeiro define o modelo de comunicação fundamental da internet (quem pede e quem responde), enquanto o segundo define como a parte que "responde" (o backend) é construída internamente para lidar com aplicações modernas e de grande escala.

Abaixo está o detalhamento de cada conceito e como eles se relacionam.

1. Arquitetura Cliente-Servidor

É o modelo clássico de computação distribuída que divide as tarefas entre os provedores de um recurso (servidores) e os requerentes desse serviço (clientes).



Cliente: É a interface com a qual o usuário interage (o navegador web, o aplicativo no celular, o software no desktop). Ele não processa regras de negócio pesadas nem armazena o banco de dados principal; sua função é capturar a ação do usuário, empacotar essa requisição e enviá-la pela rede.

Servidor: É a máquina (ou conjunto de máquinas) robusta que fica aguardando requisições. Quando recebe um pedido do cliente, ele processa as regras de negócio, consulta o banco de dados e devolve a resposta adequada.
Dinâmica: Um servidor atende a múltiplos clientes simultaneamente. Se o servidor cair, os clientes ficam inoperantes (o famoso "sistema fora do ar").

Monolito: Tradicionalmente, o "Servidor" era construído como uma aplicação monolítica: um único bloco de código gigantesco que continha todas as funcionalidades (login, carrinho de compras, pagamentos, catálogo) rodando juntas e conectadas a um único grande banco de dados.

2. Arquitetura de Microsserviços

Os microsserviços não substituem a ideia de cliente-servidor; eles revolucionam a forma como o Servidor é construído.

Em vez de ter um servidor monolítico gigantesco, a aplicação backend é dividida em dezenas (ou centenas) de serviços pequenos, independentes e focados em uma única responsabilidade.

- A Estrutura: Cada microsserviço é como se fosse um "mini-servidor" independente. Em um e-commerce, você teria um microsserviço só para gerenciar o Catálogo, outro só para o Carrinho, outro para o Pagamento e outro para Autenticação.

- Comunicação: Eles se comunicam entre si e com o cliente final através de APIs (geralmente via HTTP/REST ou mensageria).

- Bancos de Dados Independentes: Essa é a regra de ouro dos microsserviços. O serviço de Pagamento tem seu próprio banco de dados, e o serviço de Catálogo tem o dele. Se um banco cair, o outro continua funcionando.

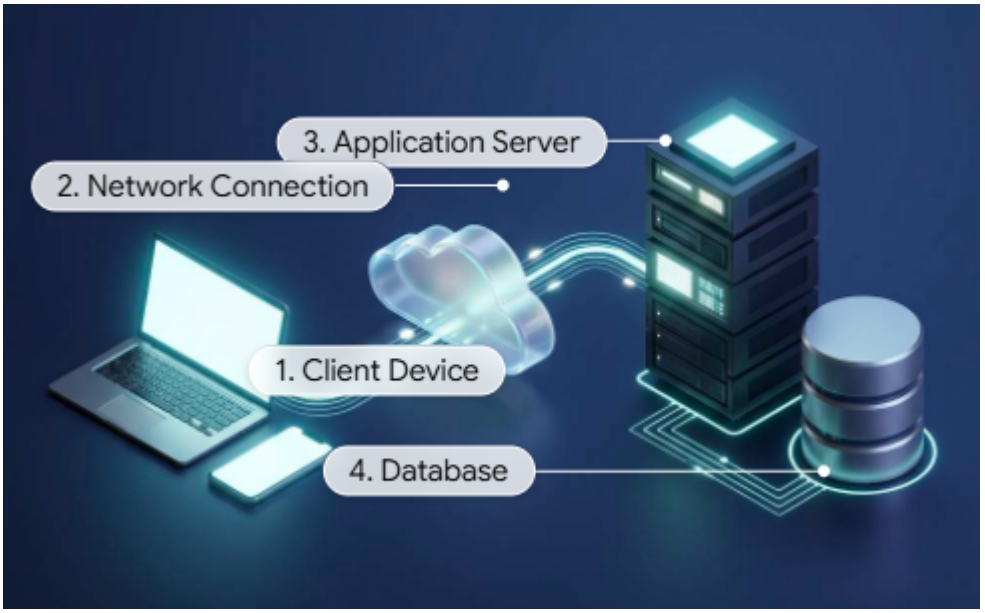
- Vantagens (Por que usar?):

    - Escalabilidade isolada: Se é Black Friday, você não precisa escalar o sistema inteiro, apenas multiplica os servidores do serviço de "Pagamento" e "Carrinho".

    - Tolerância a falhas: Se o microsserviço de "Recomendações" travar, o usuário ainda consegue finalizar a compra, pois o microsserviço de "Checkout" está intacto.

    - Independência Tecnológica: O time de Pagamento pode programar em Java com banco Oracle, enquanto o time de Recomendações usa Python com MongoDB.

Ponto-chave da evolução: Todo sistema de microsserviços opera dentro de um modelo cliente-servidor (o seu celular ainda é o cliente pedindo dados). A diferença é que, em vez de o cliente bater em um único servidor "faz-tudo", ele bate em um *API Gateway*, que atua como um maestro roteando o pedido do cliente para a rede interna de microsserviços específicos.



5.5 Design Patterns e a Gang of Four (GoF)

Origem e Conceito

O conceito de padrões teve origem no trabalho do arquiteto urbanista Christopher Alexander (*A Pattern Language*, 1977), que formalizou soluções reutilizáveis para problemas de ambiente urbano. Em 1987, Kent Beck e Ward Cunningham adaptaram essa ideia para a programação em Smalltalk. Em 1994, Erich Gamma, Richard Helm, Ralph Johnson e John Vlissides — conhecidos como a Gangue dos Quatro (Gang of Four - GoF) — publicaram o livro clássico *Design Patterns: Elements of Reusable Object-Oriented Software*, catalogando 23 padrões de projeto.

Um Design Pattern não representa código pronto ou bibliotecas reutilizáveis, mas uma descrição ou modelo (*template*) genérico de como resolver um problema frequente em projeto orientado a objetos.

Estrutura Obrigatória de um Padrão GoF

Todo padrão de projeto GoF deve atender a 4 elementos essenciais:

- **Nome:** Um identificador conciso que compõe o vocabulário técnico comum entre os desenvolvedores.

- **Problema:** O contexto, os sintomas e as forças que indicam quando o padrão deve ser aplicado.

- **Solução:** A descrição abstrata dos elementos de projeto, seus relacionamentos, responsabilidades e colaborações.

- **Consequências:** Os resultados, custos, benefícios e trade-offs decorrentes do uso do padrão.

Categorização dos 23 Padrões GoF

Os 23 padrões da GoF dividem-se em 3 famílias principais:

1. **Padrões Criacionais:** Abstraem e adiam a criação e instanciação de objetos, favorecendo a composição dinâmica em relação à herança rígida. (*Abstract Factory, Builder, Factory Method, Prototype, Singleton*).

2. **Padrões Estruturais:** Focam na composição de classes e objetos para construir estruturas maiores e flexíveis em tempo de execução. (*Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy*).

3. **Padrões Comportamentais:** Concentram-se na atribuição de responsabilidades, algoritmos e padrões de comunicação entre objetos. (*Chain of Responsibility, Command, Interpreter, Iterator, Mediator, Memento, Observer, State, Strategy, Template Method, Visitor*).

Exemplos Práticos

- **Factory Method (Criacional):** Encapsula a lógica de criação de objetos em subclasses ou fábricas concretas, evitando que o código cliente fique acoplado a implementações específicas (por exemplo, instanciar pagamentos via Cartão ou Pix sem múltiplos blocos *if/switch*).

- **Adapter (Estrutural):** Converte a interface de um componente ou API externa na interface esperada pelos clientes da aplicação, permitindo a integração de código legado sem refatorações drásticas.
- **Strategy (Comportamental):** Encapsula uma família de algoritmos intercambiáveis (por exemplo, regras de cálculo de frete por Correios ou Transportadora), permitindo alterar o comportamento em tempo de execução.

Perspectiva Arquitetural dos Padrões de Projeto

Embora projetados para orientação a objetos, padrões como *Observer*, *Composite*, *Interpreter*, *Mediator* e *Proxy* desempenham papéis estratégicos na arquitetura global do sistema. Nesses cenários, eles funcionam como táticas arquiteturais para suportar atributos de qualidade e resolver conflitos de requisitos.

5.6 Architecture Decision Records (ADR) e Análise de Trade-offs

O que é uma ADR e o Modelo Debt-Aware

Um Architecture Decision Record (ADR) é um documento leve e conciso que captura uma decisão arquitetural significativa (ASR), seu contexto motivador, opções descartadas e consequências operacionais. Proposto originalmente por Michael Nygard em 2011 e adotado por especialistas como Martin Fowler e diretrizes como o Microsoft Azure Well-Architected Framework, o modelo de ADRs consciente de débitos (*Debt-Aware*) transforma obrigações e riscos ocultos em escolhas explícitas, controladas e mensuráveis.

Princípio Append-Only e Governança

O repositório de decisões arquiteturais atua como um histórico imutável (*append-only*) armazenado junto ao código-fonte (em diretórios como doc/adr) em linguagens de marcação leves como Markdown:

- **Imutabilidade:** Registros aceitos não devem ser editados posteriormente quando as circunstâncias mudam.

- **Substituição (Superseding):** Caso uma decisão anterior precise ser alterada, cria-se um novo registro que substitui (*supersedes*) a ADR antiga e estabelece um link entre ambas, preservando a rastreabilidade histórica e evitando discussões repetitivas.

Estrutura Básica de uma ADR

Uma ADR típica é composta por:

- **Título (Title):** Nome declarativo e numerado de forma sequencial e cronológica.

- **Data (Date) e Status:** Estado atual da decisão (ex.: *Proposto*, *Aceito*, *Rejeitado* ou *Substituído*).
- **Contexto (Context):** Descrição do problema, forças de negócio, ambiente e restrições técnicas que motivaram a escolha.

- **Decisão (Decision):** A solução adotada e a listagem explícita das alternativas consideradas e descartadas.

- **Consequências (Consequences / Trade-offs):** Análise dos impactos positivos, limitações e débitos técnicos conscientemente aceitos.

Formato Y-Statement e Story Slide

Para padronizar a escrita sintética de decisões, utiliza-se o modelo Y-Statement: *"No contexto de [Serviço/Sistema], enfrentando a necessidade de [Problema/Pressão], decidimos por [Decisão] (e não [Alternativas descartadas]) para alcançar [Resultado/Métrica], aceitando que [Trade-offs/Débitos Assumidos]"*. Para a comunicação com comitês executivos e partes interessadas de negócio, a estrutura é traduzida no formato de narrativa (*Decision Story*).

A Primeira Lei da Arquitetura e Análise de Trade-offs

A Primeira Lei da Arquitetura de Software estabelece que *não existem soluções perfeitas em arquitetura de software, apenas trade-offs*. Cada ganho em um atributo de qualidade exige concessões em outros:

- **Desempenho vs. Modificabilidade:** A adição de níveis de indireção e camadas de abstração (*Layers*, *Reflection*, *Microkernel*) eleva a modificabilidade e modularidade, mas introduz overhead de comunicação e aumenta a latência de execução.

- **Desempenho vs. Confiabilidade/Disponibilidade:** Táticas como replicação de dados e nós para tolerância a falhas garantem alta disponibilidade, mas consomem mais recursos computacionais e reduzem o tempo de resposta.

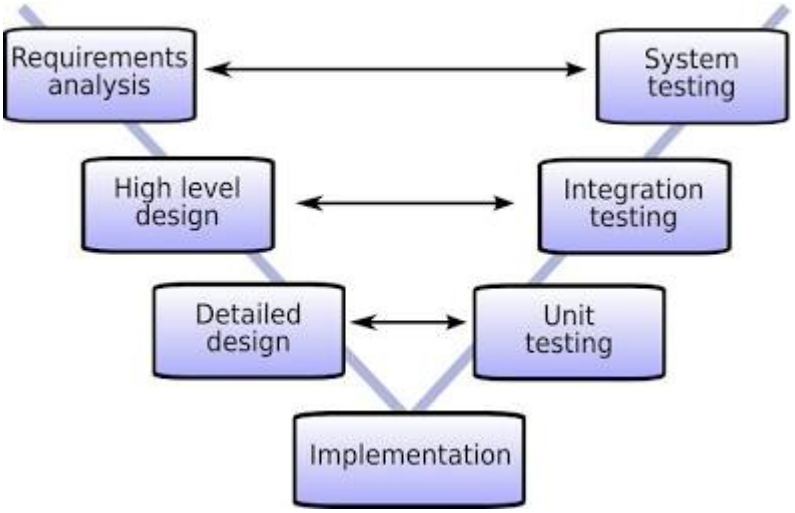
- **Proxy como Resolvedor de Conflito:** O padrão *Proxy* pode intermediar objetos pesados (carregamento sob demanda) para melhorar o desempenho do cliente sem comprometer a simplicidade e a modificabilidade do código.

MÓDULO 6 — Verificação, Validação e Testes

6.1 Verificação e Validação

A diferença fundamental entre esses dois conceitos, muito utilizados em Engenharia de Software e Gestão da Qualidade, resume-se a duas perguntas clássicas:

- **Verificação:** "Estamos construindo o produto **corretamente**?"
- **Validação:** "Estamos construindo o **produto certo**?"



Embora andem juntos, eles avaliam etapas e propósitos completamente diferentes no ciclo de vida de um projeto.

Verificação (O Processo)

A verificação é um processo interno e técnico. Ela garante que o software, sistema ou produto está sendo desenvolvido de acordo com as especificações, requisitos e padrões técnicos definidos previamente, sem desvios.

- **Foco:** No **processo** de desenvolvimento e na arquitetura.
- **Quando ocorre:** Durante a construção, antes que o produto final esteja pronto (fases iniciais e intermediárias).
- **Como é feita:** Através de revisões de código, inspeções de documentos, análises estáticas, testes de unidade (unitários) e testes de integração.
- **Quem participa:** Desenvolvedores, arquitetos de software e equipes técnicas de qualidade (QA). O cliente geralmente não participa desta etapa.

Validação (O Produto Final)

A validação é um processo externo e voltado para o negócio. Ela prova que o produto final realmente resolve o problema do cliente e atende às suas necessidades reais no ambiente em que será utilizado.

- **Foco:** No **produto final** e na experiência do usuário.
- **Quando ocorre:** Após a conclusão do desenvolvimento de uma funcionalidade ou do produto como um todo (fases finais).
- **Como é feita:** Através de testes de aceitação do usuário (UAT), testes beta, testes de sistema e validação em ambiente de produção (ou simulado).
- **Quem participa:** Usuários finais, clientes (Product Owners) e testadores focados na regra de negócio.

Comparativo Direto

Característica	**Verificação**	**Validação**
Pergunta-chave	O sistema segue as especificações técnicas?	O sistema resolve o problema do usuário?
Natureza	Avaliação estática (leitura de código/docs) e testes de baixo nível.	Avaliação dinâmica (executar o software real).
Alvo principal	Encontrar bugs na lógica e falhas de arquitetura.	Encontrar falhas na regra de negócio e usabilidade.
Referência	Documento de requisitos, design, diagramas UML.	Necessidades reais do cliente e mercado.

Um Exemplo Prático (Construindo uma Casa)

Imagine que você contratou uma empreiteira para construir uma casa.
A **Verificação** acontece quando o engenheiro confere se o cimento usado é da marca correta, se as vigas suportam o peso calculado na planta e se a fiação

elétrica segue as normas de segurança (NR-10). Tudo está sendo construído *corretamente* segundo o manual.

A **Validação** acontece quando você (o cliente) entra na casa pronta e percebe que as portas são estreitas demais para passar a sua geladeira e que a janela do quarto bate direto no muro do vizinho, bloqueando o sol. A casa foi construída exatamente como estava na planta (passou na verificação), mas não atende à sua necessidade real de moradia (falhou na validação).

6.2 Testes unitários, integração, sistema e aceitação

Os **testes unitários, de integração, de sistema e de aceitação** representam os quatro níveis fundamentais da pirâmide de testes no desenvolvimento de software, evoluindo do nível mais técnico e granular (código) até o nível funcional e estratégico (negócio).

1. Testes Unitários (Unit Testing)

Avaliam a menor unidade isolada de código da aplicação (como uma função, método ou classe) para garantir que ela funcione exatamente como projetada.

- **Foco:** Verificação técnica da lógica interna.
- **Quem executa:** Desenvolvedores (geralmente automatizado durante a escrita do código).
- **Isolamento:** Uso de *mocks* ou *stubs* para simular dependências externas (bancos de dados, APIs de terceiros).
- **Exemplo:** Testar se uma função calcularDesconto(valor, taxa) retorna 90 quando recebe 100 e 0.10.

2. Testes de Integração (Integration Testing)

Avaliam a comunicação e a troca de dados entre duas ou mais unidades/módulos do sistema previamente testados em separado.

- **Foco:** Verificação das interfaces, contratos e integração entre componentes.
- **Quem executa:** Desenvolvedores e Engenheiros de QA.
- **Ambiente:** Requer dependências reais ou simuladores avançados (ex.: banco de dados local ou contêineres).

- **Exemplo:** Verificar se a API de pedidos consegue gravar com sucesso os dados da compra no banco de dados PostgreSQL e enviar uma mensagem para a fila do RabbitMQ.

3. Testes de Sistema (System Testing)

Avaliam o sistema completo e totalmente integrado de ponta a ponta em um ambiente que simula a produção.

- **Foco:** Transição entre Verificação e Validação; testa requisitos funcionais (fluxos de ponta a ponta) e não funcionais (desempenho, segurança, usabilidade).

- **Quem executa:** Equipe de QA / Testadores dedicados.

- **Ambiente:** Staging ou Homologação idêntico/similar ao de produção.

- **Exemplo:** Fazer o login, adicionar um produto ao carrinho, efetuar o pagamento via PIX, verificar se o pedido muda para "pago" e se o e-mail de confirmação é disparado.

4. Testes de Aceitação (Acceptance Testing / UAT)

Avaliam se a solução atende aos requisitos de negócio e às expectativas de uso do cliente final antes do lançamento definitivo.

- **Foco:** Validação do produto ("Resolve o problema do usuário?").

- **Quem executa:** Cliente, Product Owner (PO), stakeholders ou usuários finais (em testes Alpha/Beta).

- **Objetivo:** Obter a aprovação formal do cliente para liberação em produção.

- **Exemplo:** O gerente da loja utiliza a plataforma em homologação e confirma que os relatórios gerados atendem exatamente às regras operacionais do setor contábil da empresa.

Comparativo Rápido

Nível	Foco Principal	Verificação ou Validação?	Quem Executa?
Unitário	Lógica de componentes isolados	Verificação	Desenvolvedores
Integração	Comunicação entre módulos e APIs	Verificação	Devs / QA
Sistema	Comportamento do produto ponta a ponta	Verificação + Validação	QA / Testadores
Aceitação	Requisitos do negócio e usabilidade	Validação	Cliente / PO / Usuário

6.3 Caixa-preta e caixa-branca

Os testes de **Caixa-Branca** e **Caixa-Preta** são duas abordagens complementares de engenharia de software definidas pelo nível de acesso e conhecimento que o testador possui sobre o código-fonte e a arquitetura interna do sistema.

1. Teste de Caixa-Branca (Glass-Box ou Teste Estrutural)

No teste de caixa-branca, o sistema é tratado como um recipiente transparente. O testador tem acesso total ao código-fonte, diagramas de arquitetura e estruturas de dados, focando na **lógica interna e no fluxo de execução**.

- **Foco:** "Como o sistema foi construído e se sua estrutura interna está correta."
- **Quem executa:** Desenvolvedores e Engenheiros de Testes de Software (SDETs).

- **Níveis mais comuns:** Testes Unitários e Testes de Integração técnica.

Principais Técnicas de Caixa-Branca

- **Cobertura de Comandos (Statement Coverage):** Garante que cada linha individual do código seja executada pelo menos uma vez durante a suíte de testes.

- **Cobertura de Decisões/Desvios (Branch/Decision Coverage):** Valida todos os caminhos possíveis originados por estruturas condicionais (if/else, switch, loops for/while).

- **Cobertura de Condição e Caminho (Path Coverage):** Analisa todas as combinações possíveis de condições booleanas e caminhos de execução de um método.

- **Análise de Complexidade Ciclomática:** Métrica matemática usada para medir a quantidade de caminhos independentes no código, indicando quais módulos são muito complexos e propensos a falhas.

Vantagens e Desvantagens

- **Vantagens:** Identifica código morto (nunca executado), otimiza o desempenho, descobre vulnerabilidades de segurança (ex: brechas para SQL Injection) e garante alta cobertura de código.

- **Desvantagens:** Alto custo operacional; exige conhecimento profundo de programação; não garante que o software atenda às expectativas do cliente (pode testar um código perfeito que executa uma regra de negócio errada).

2. Teste de Caixa-Preta (Teste Funcional ou Baseado em Especificações)

No teste de caixa-preta, o sistema é tratado como uma caixa opaca. O testador não tem acesso ao código-fonte e interage com a aplicação fornecendo dados de entrada (**inputs**) e validando os resultados produzidos (**outputs**) em relação aos requisitos especificados.

- **Foco:** "O que o sistema faz e se atende às regras de negócio e necessidades do usuário."

- **Quem executa:** Analistas de QA, Analistas de Negócio e Usuários Finais (UAT).

- **Níveis mais comuns:** Testes de Sistema e Testes de Aceitação.

Principais Técnicas de Caixa-Preta

- **Particionamento por Equivalência:** Divide o conjunto de dados de entrada em classes válidas e inválidas. Testa-se apenas um representante de cada classe para otimizar o tempo.

- **Análise do Valor Limite (Boundary Value Analysis):** Foca os testes nas bordas dos limites permitidos. Se um sistema aceita idades de 18 a 60 anos, os testes avaliam exatamente os pontos críticos: 17, 18, 19 e 59, 60, 61.

- **Tabelas de Decisão:** Matrizes estruturadas para testar combinações complexas de regras de negócio e suas respectivas ações esperadas.

- **Transição de Estados:** Avalia como o sistema reage ao passar de um estado para outro (ex.: *Aguardando Pagamento* $\rightarrow$ *Pagamento Aprovado* $\rightarrow$ *Enviado*).

Vantagens e Desvantagens

- **Vantagens:** Foco direto na experiência do usuário e nas regras de negócio; independe do conhecimento de linguagens de programação; os testes são imparciais pois não sofrem viés de quem escreveu o código.

- **Desvantagens:** Não consegue testar caminhos ocultos ou não documentados no código; risco de testes redundantes; difícil identificar a causa raiz exata de uma falha apenas olhando a interface.

3. O Elo Intermediário: Teste de Caixa-Cinzenta (Gray-Box)

Existe uma abordagem híbrida chamada **Caixa-Cinzenta**, onde o testador possui conhecimento parcial da estrutura interna.

O testador não altera o código-fonte, mas tem acesso a documentações de API (como Swagger/OpenAPI), schemas de banco de dados, arquivos de log ou arquitetura de microsserviços. É amplamente utilizada em testes de API, integração de sistemas e testes de segurança de penetração (Pentests).

Comparativo Direto

Critério	Caixa-Branca	Caixa-Preta	Caixa-Cinzenta
Visibilidade do Código	Total	Nenhuma	Parcial (APIs, DB, Logs)
Conhecimento Exigido	Programação e arquitetura	Regras de negócio e uso	Estrutura de APIs e dados
Base para Criação dos Testes	Código-fonte e algoritmos	Requisitos e Casos de Uso	Schemas, APIs e arquitetura
Principal Objetivo	Qualidade do código e estrutura	Qualidade do produto e uso	Integração e integridade de dados
Nível do Modelo V	Teste Unitário	Teste de Aceitação e Sistema	Teste de Integração e API

Exemplo Prático Integrado: Tela de Login com Bloqueio de Segurança

Imagine um sistema que bloqueia o usuário após 3 tentativas incorretas de senha:

- **Teste de Caixa-Branca:** O desenvolvedor inspeciona a função autenticar(), verifica se a biblioteca de hash (BCrypt) é invocada corretamente, se a instrução SQL previne injeções de código malicioso e se o contador em memória incrementa adequadamente até disparar a exceção de bloqueio.

- **Teste de Caixa-Cinzenta:** O testador faz requisições via Postman para o endpoint /api/login, inspeciona os códigos de retorno HTTP (401 Unauthorized e 429 Too Many Requests) e executa uma consulta direta no banco de dados para checar se a flag is_locked mudou para true.

- **Teste de Caixa-Preta:** O testador de QA abre o navegador, digita o usuário, insere senhas erradas 3 vezes diretamente na tela e valida se a mensagem de alerta *"Sua conta foi bloqueada por 15 minutos"* aparece visível na interface do usuário.

6.4 Casos de Teste (Test Cases)

Um **Caso de Teste** é um documento formal ou especificação que descreve um conjunto de condições, variáveis, entradas, passos de execução e resultados esperados desenvolvidos para verificar se uma funcionalidade específica do software está funcionando conforme o projetado. ele serve como o "roteiro" que o testador (ou o script) deve seguir para validar uma regra de negócio ou requisito técnico.

Estrutura Fundamental de um Caso de Teste

Para ser eficaz, um caso de teste deve ser claro, reproduzível, independente de outros testes e rastreável a um requisito. Os campos essenciais incluem:

Campo	**Descrição**	**Exemplo**
ID do Teste	Identificador único	CT-AUT-001
Título / Descrição	Resumo objetivo do que está sendo testado	Validar bloqueio de login com senha incorreta
Pré-condições	Estado do sistema antes da execução	Usuário cadastrado e ativo no sistema
Massa de Dados (Inputs)	Dados específicos usados no teste	Usuário: joao@email.com / Senha: SenhaErrada123
Passos de Execução	Sequência detalhada de ações	1. Acessar tela de login 2. Preencher e-mail 3. Preencher senha incorreta 4. Clicar em "Entrar"
Resultado Esperado	Comportamento correto do sistema	Alerta: "Usuário ou senha inválidos" e permanece

		na página
Pós-condições	Estado do sistema após a execução	Nenhuma alteração no estado do perfil do usuário

6.5 Testes Automatizados (Automated Testing)

A **Automação de Testes** consiste no uso de ferramentas e scripts de software para executar casos de teste de forma automática, sem a necessidade de intervenção humana direta a cada execução. O objetivo primário é aumentar a velocidade de feedback, reduzir erros humanos em tarefas repetitivas e garantir a qualidade contínua do produto durante entregas frequentes (Integração e Entrega Contínuas - CI/CD).

Quando Automatizar vs. Quando Manter Manual

Nem todo teste deve ser automatizado. A decisão depende do Retorno sobre o Investimento (ROI):

- **Excelente para Automação:**
  - **Testes de Regressão:** Validação de funcionalidades antigas que precisam ser executadas a cada novo deploy.
  - **Testes de fumaça (Smoke Tests):** Testes rápidos para checar se as funções vitais do sistema estão operantes após uma alteração.
  - **Testes de Carga e Desempenho:** Simulação de milhares de usuários simultâneos (impossível de realizar manualmente).
  - **Testes Orientados a Dados (Data-Driven):** Executar a mesma funcionalidade com centenas de dados de entrada diferentes.
- **Melhor Manter Manual:**
  - **Testes Exploratórios:** Onde a intuição, criatividade e experiência do testador buscam cenários não mapeados.
  - **Testes de Usabilidade e UX:** Avaliação de quão agradável, intuitivo ou acessível o sistema é para uma pessoa real.

- **Funcionalidades Instáveis/Incipientes:** Recursos em fase inicial de design que mudam constantemente de layout ou regra (o custo de manutenção do código de teste será maior que a execução manual).

Ferramentas Populares do Mercado

- **APIs / Backend:** Postman, RestAssured, Supertest.
- **Web / Interface:** Cypress, Playwright, Selenium.
- **Mobile:** Appium.
- **Desempenho / Carga:** JMeter, k6, Locust.

6.6 Defeitos e Rastreamento (Defect & Bug Tracking)

Um **Defeito** (bug, falha ou anomalia) é qualquer desvio no software entre o comportamento observado e o comportamento esperado especificado nos requisitos ou desejado pelo usuário.

O **Rastreamento de Defeitos** é o processo de registrar, organizar, priorizar e monitorar o status de uma falha desde a sua descoberta por um testador até a sua correção e validação final pelo time de engenharia.

O Ciclo de Vida do Defeito

O caminho que uma falha percorre no fluxo de trabalho geralmente segue este padrão:

1. **Novo (Novo/Aberto):** O teste falhou e o QA registra o defeito no sistema de rastreamento.
2. **Atribuído (Assigned):** O líder de desenvolvimento ou gerente analisa a falha e a atribui a um desenvolvedor específico.
3. **Em Correção (In Progress):** O desenvolvedor investiga o código-fonte e escreve a correção.
4. **Resolvido/Corrigido (Resolved/Fixed):** A correção foi enviada para o ambiente de testes (Staging/Homologação).
5. **Re-teste (Pending Retest):** O QA executa novamente o caso de teste que gerou a falha original.

6. **Fechado (Closed):** A correção funcionou e o bug é oficialmente encerrado.

   ○ *Alternativa:* **Reaberto (Reopened)** caso o defeito persistir durante o re-teste.

Severidade vs. Prioridade

A classificação correta de defeitos evita conflitos entre as equipes de QA, Desenvolvimento e Negócio:

- **Severidade (Impacto Técnico):** Mede o grau de dano que a falha causa ao funcionamento do sistema.

  ○ *Exemplo de Severidade Alta:* Um erro de banco de dados faz o sistema cair completamente (*crash*) ao tentar salvar um registro.

- **Prioridade (Urgência de Negócio):** Mede a rapidez com que a falha deve ser corrigida com base nas necessidades comerciais ou operacionais.

  ○ *Exemplo de Prioridade Alta / Severidade Baixa:* O logotipo da empresa aparece trocado ou com o nome digitado errado na página principal de login. Não quebra o sistema (baixa severidade), mas prejudica a marca e precisa ser corrigido imediatamente (alta prioridade).

Anatomia de um Bom Relatório de Bug (Bug Report)

Um relatório de defeito de alta qualidade deve conter:

- **Título claro:** *Ex: [Checkout] Erro 500 ao tentar finalizar compra usando cartão de crédito da bandeira Visa.*

- **Passos para reproduzir:** Lista numerada exata de como refazer a falha.

- **Resultado Obtido vs. Resultado Esperado:** O que aconteceu x O que deveria ter acontecido.

- **Ambiente:** Navegador/Dispositivo, versão do SO, versão da build/aplicação.

- **Evidências:** Capturas de tela (screenshots), gravações de vídeo e logs de console/rede.

Ferramentas de Rastreamento Comuns

Jira, Azure DevOps, Bugzilla, Redmine e Trello.

MÓDULO 7 — UML e Modelagem de Sistemas

7.1 Diagrama caso de uso

O que é diagrama de caso de uso?

Na linguagem de modelagem unificada (UML), o diagrama de caso de uso resume os detalhes dos usuários do seu sistema (também conhecidos como atores) e as interações deles com o sistema. Para criar um, use um conjunto de símbolos e conectores especializados. Um bom diagrama de caso de uso ajuda sua equipe a representar e discutir:

- Cenários em que o sistema ou aplicativo interage com pessoas, organizações ou sistemas externos
- Metas que o sistema ou aplicativo ajuda essas entidades (conhecidas como atores) a atingir
- O escopo do sistema

O que é um diagrama de casos de uso em UML?

Um caso de uso reúne cenários. Ao descrever as partes principais do seu processo, incluindo caminhos alternativos, você terá uma visão mais clara do escopo do seu projeto.

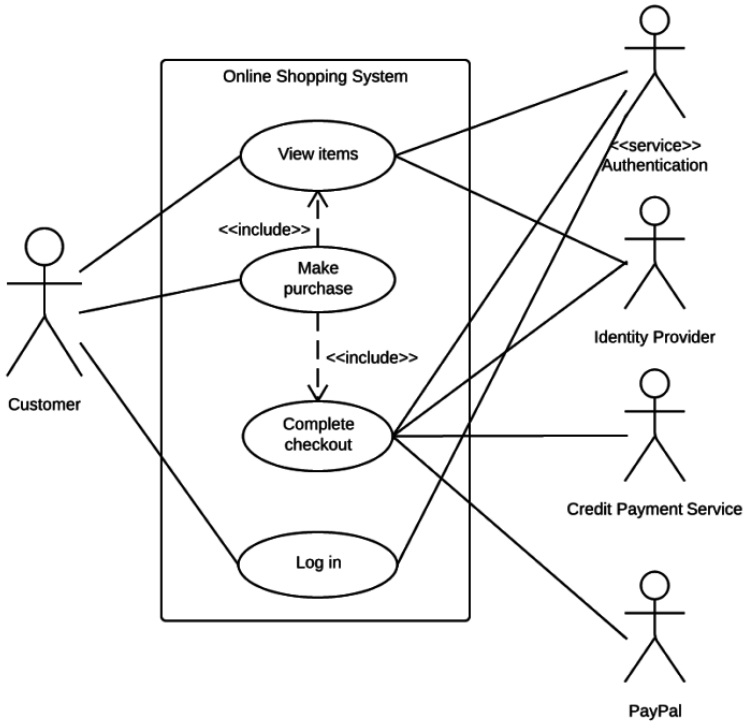
Quando usar o diagrama de caso de uso

O diagrama de caso de uso não oferece muitos detalhes, não espere, por exemplo, que ele mostre a ordem em que os passos são executados. Em vez disso, um diagrama de caso de uso adequado dá uma visão geral do relacionamento entre casos de uso, atores e sistemas. Os especialistas recomendam usar o diagrama de caso de uso para complementar um caso de uso descrito em texto.

UML é o kit de ferramentas de modelagem para criar o diagrama. O caso de uso é representado por uma forma oval rotulada. Bonecos palito representam os atores no processo, e a participação do ator no sistema é modelada com uma linha entre o ator e o caso de uso. Para representar o limite do sistema, desenhe uma caixa em torno do próprio caso de uso.

O diagrama de caso de uso UML é ideal para:

- Representar as metas de interações entre sistemas e usuários
- Definir e organizar requisitos funcionais no sistema
- Especificar o contexto e os requisitos do sistema
- Modelar o fluxo básico de eventos no caso de uso



Símbolos e notação no diagrama de caso de uso

A notação do diagrama de caso de uso é bastante objetiva e não envolve a mesma quantidade de símbolos de outros diagramas UML.

- **Caso de uso:** formato oval na horizontal e que representam os diferentes usos que um usuário pode ter.
- **Atores:** bonecos palito, representando as pessoas que realmente implementam os casos de uso.
- **Associações:** uma linha entre atores e casos de uso. Nos diagramas complexos, é importante saber quais atores estão associados a quais casos de uso.

- **Caixa de limite do sistema:** caixa que define um escopo do sistema para os casos de uso. Todos os casos de uso fora da caixa são considerados fora do escopo do sistema. Por exemplo: o Psycho Killer está fora do escopo de ocupações no exemplo da serra elétrica abaixo.

- **Pacote:** uma forma UML na qual colocar diferentes elementos em grupos. Assim como no diagrama de componentes, esses agrupamentos são representados como pastas de arquivos. Assim como no diagrama de componentes, esses agrupamentos são representados como pastas de arquivos.

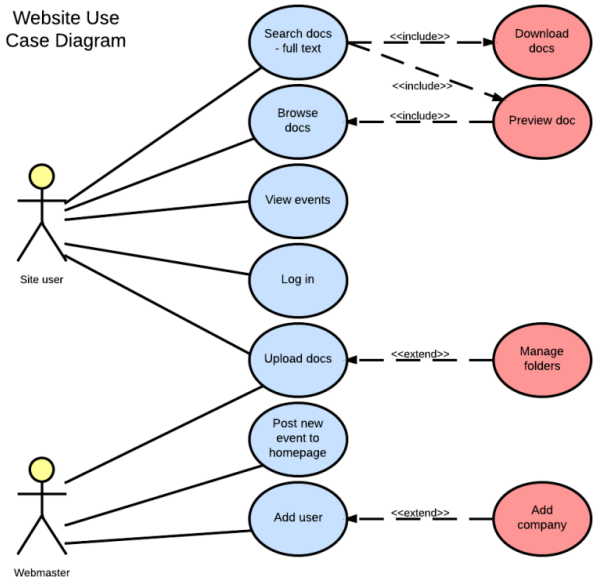
Componentes do diagrama de caso de uso

Para entender o que é um diagrama de caso de uso, é necessário primeiro saber como ele é feito. São componentes comuns:

- **Atores:** os usuários que interagem com o sistema. Ator pode ser uma pessoa, organização ou sistema externo que interage com seu aplicativo ou sistema. Eles devem ser objetos externos que produzem ou consomem dados.

- **Sistema:** uma sequência específica de ações e interações entre os atores e o sistema. O sistema também pode ser chamado de cenário.

- **Metas:** o resultado final da maioria dos casos de uso. Um diagrama criado corretamente deve descrever as atividades e variantes usadas para atingir a meta.



7.2 Diagrama de Classes

O que é um diagrama de classes UML?

A UML (do inglês, Unified Modeling Language) pode ajudar você a modelar sistemas de várias maneiras. Um dos tipos mais populares na UML é o diagrama de classes. Na taxonomia mais ampla da UML, os diagramas de classes se enquadram na família de diagramas estruturais (estáticos), que se concentram no que um sistema é feito, e não em como ele se comporta ao longo do tempo.

A UML foi estabelecida como um modelo padronizado para descrever uma abordagem de programação orientada a objetos. Como as classes são os blocos de construção dos objetos, os diagramas de classes são os blocos de construção da UML. Os vários componentes em um diagrama de classes podem representar as classes que serão programadas de fato, os objetos principais ou as interações entre classes e objetos. Como se alinham estreitamente com a programação orientada a objetos, os diagramas de classes geralmente podem ser traduzidos diretamente em construtos de código em linguagens OOP comuns, como Python, Java e C++.

A forma da classe em si consiste em um retângulo com três linhas. A linha superior contém o nome da classe, a linha central contém os atributos da classe e a seção inferior expressa os métodos ou operações que a classe pode usar. Classes e subclasses são agrupadas para mostrar o relacionamento estático entre cada objeto.

Benefícios dos diagramas de classes

Os diagramas de classes oferecem vários benefícios para qualquer organização. Use diagramas de classes UML para:

- Ajudar os desenvolvedores a pensar na estrutura e nas interações das classes antes de escrever qualquer código. Isso pode resultar em um software mais limpo e com melhor funcionamento.

- Desenhar gráficos detalhados e documentar requisitos que precisam ser incluídos no código.

- Compartilhar informações com as partes interessadas de forma mais fácil com apresentações visuais, sem precisar se aprofundar no código.

- Ilustrar modelos de dados para qualquer sistema, por mais complexo que seja.

- Compartilhar facilmente informações em toda a sua empresa.

- Ter uma melhor compreensão dos esquemas gerais de um aplicativo.

- Documentar requisitos e outros aspectos de um sistema sem explicações complicadas e cheias de texto.

- Criar modelos a partir de uma perspectiva de negócios como analista de negócios.

Eles também são úteis no início do ciclo de vida como uma ferramenta de reflexão ajudando as equipes a raciocinar sobre a estrutura e os relacionamentos antes da implementação e como um artefato de comunicação para partes interessadas técnicas e não técnicas.

Componentes básicos de um diagrama de classes

Dependendo do contexto, as classes em um diagrama de classes podem representar os objetos principais, as interações no aplicativo ou as classes a serem programadas. Para responder à pergunta "O que é um diagrama de classes na UML?", você deve primeiro entender sua composição básica.

- **Classes:** um modelo para criar objetos e implementar comportamento em um sistema. Na UML, uma classe representa um objeto ou um conjunto de objetos que compartilham uma estrutura e um comportamento comuns. Elas são representadas por um retângulo que inclui linhas com o nome da classe, seus atributos e suas operações. Ao desenhar uma classe em um diagrama de classes, você só precisa preencher a linha superior as outras são opcionais caso queira fornecer mais detalhes.

- **Sinais:** símbolos que representam comunicações assíncronas e unidirecionais entre objetos ativos.

- **Tipos de dados:** classificadores que definem valores de dados. Os tipos de dados podem modelar tanto tipos primitivos quanto enumerações.

- **Pacotes:** formas projetadas para organizar classificadores relacionados em um diagrama. Eles são simbolizados por uma forma de retângulo grande com guias.

- **Interfaces:** uma coleção de assinaturas de operação e/ou definições de atributos que definem um conjunto coeso de comportamentos. As interfaces são semelhantes às classes, exceto que uma classe pode ter

uma instância do seu tipo, e uma interface deve ter pelo menos uma classe para implementá-la.

- **Enumerações:** representações de tipos de dados definidos pelo usuário. Uma enumeração inclui grupos de identificadores que representam valores da enumeração.

- **Objetos:** instâncias de uma ou mais classes. Os objetos podem ser adicionados a um diagrama de classes para representar instâncias concretas ou prototípicas.

- **Artefatos:** elementos de modelo que representam as entidades concretas em um sistema de software, como documentos, bancos de dados, arquivos executáveis, componentes de software etc.

Seções de uma classe

A classe padrão é posta por três seções:

Seção superior

Contém o nome da classe. Esta seção é sempre obrigatória, seja falando do classificador ou de um objeto. O nome da classe deve:

- Começar com letra maiúscula.
- Estar formatado em negrito.
- Estar centralizado na seção superior.
- Ser descritivo.

Seção central

Contém os atributos da classe. Use esta seção para descrever as qualidades da classe. Isso só é obrigatório ao descrever uma instância específica de uma classe.

Seção inferior

Inclui as operações (métodos) da classe. Exibida em formato de lista, cada operação ocupa sua própria linha. As operações descrevem como uma classe interage com os dados. O texto nas duas seções inferiores geralmente é alinhado à esquerda e digitado com as primeiras letras minúsculas.

Modificadores de acesso de membros

Todas as classes têm níveis de acesso diferentes dependendo do modificador de acesso (visibilidade). Aqui estão os níveis de acesso com seus respectivos símbolos:

- Público (+)
- Privado (-)
- Protegido (#)
- Pacote (~)
- Derivado (/)
- Estático (sublinhado)

Escopos de membros

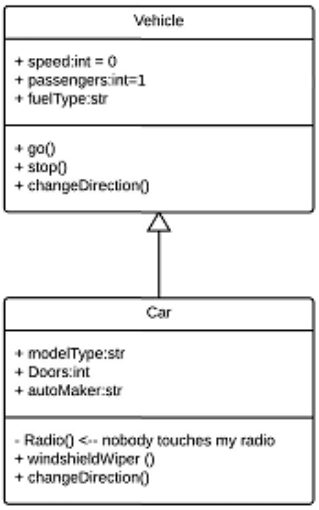
Existem dois escopos para membros: classificadores e instâncias.

Os classificadores costumam ser reconhecidos como estáticos, o que significa que os valores dos atributos permanecem os mesmos em todas as instâncias e a invocação do método não afeta o estado da instância. Em comparação, às instâncias contém métodos que podem alterar o estado da instância e valores de atributos que podem variar entre as instâncias. Para indicar um classificador, basta sublinhar seu nome; deixe o texto sem formatação para indicar uma instância.

Interações

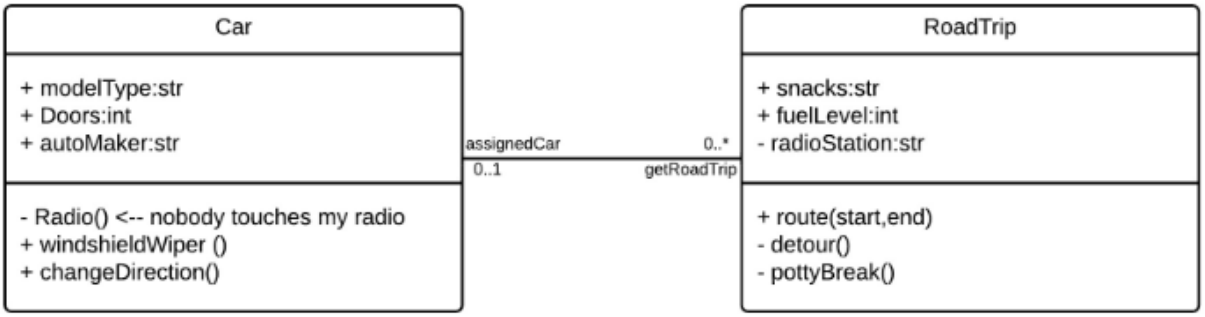
O termo "interações" refere-se aos vários relacionamentos e links que podem existir em diagramas de classes e objetos. Algumas das interações mais comuns incluem:

Herança: o processo de uma classe filha ou subclasse assumir a funcionalidade de uma classe pai ou superclasse, também conhecida como generalização. É simbolizada por uma linha reta conectada com uma ponta de seta fechada apontando para a superclasse.



Neste exemplo, o objeto "Carro" herdaria todos os atributos (velocidade, número de passageiros e combustível) e métodos (go(), stop(), changeDirection()) da classe pai ("Veículo"), além dos atributos específicos (tipo de modelo, número de portas e fabricante) e métodos de sua própria classe (Radio(), windshieldWiper(), ac/heat()). A herança é mostrada em um diagrama de classes usando uma linha sólida com uma seta fechada e vazia.

Associação bidirecional: o relacionamento padrão entre duas classes. Ambas as classes têm conhecimento uma da outra e de seu relacionamento com a outra. Essa associação é representada por uma linha reta entre duas classes.



Como exemplo, em sua viagem pelo Arizona, você pode se deparar com um radar de velocidade onde uma câmera registra sua atividade de direção, mas você não saberá disso até receber uma notificação pelo correio. Não está desenhado na imagem, mas, neste caso, o valor da multiplicidade seria 0..*, dependendo de quantas vezes você passar pelo radar.

Dicas para planejar diagramas de classes claros

Antes de começar a diagramar (or ao iterar em um rascunho inicial), alguns hábitos simples de planejamento podem tornar seus diagramas de classes mais claros e úteis para outras pessoas.

- O nome da classe deve ser descritivo e ter significado. Por exemplo, se você nomear uma classe como UserAccount, ficará claro que a classe criará algum tipo de conta de usuário.

- Identificar claramente a responsabilidade de cada classe.

- Identificar cada elemento com antecedência e entender como eles se relacionam entre si.

- Adicionar apenas propriedades relevantes a cada classe. A adição de propriedades desnecessárias pode fazer com que seu diagrama pareça poluído e excessivamente complicado.

- Usar notas breves quando precisar descrever uma parte específica do diagrama.

- Colaborar com os membros da equipe e retrabalhar o diagrama conforme necessário para ajustá-lo.

Essas práticas são especialmente úteis porque os diagramas de classes raramente existem de forma isolada; eles normalmente representam um conjunto de classes que devem trabalhar juntas como parte de um sistema maior. Embora você possa estar diagramando apenas uma única classe, é provável que ela seja usada em um sistema maior. Um sistema é representado em um diagrama de classes como uma coleção de classes que precisam trabalhar juntas. Por isso é importante identificar os elementos com antecedência e entender como eles se relacionam entre si. Quando várias classes são agrupadas em um único diagrama de classes, isso ajuda a entender a estrutura estática e os relacionamentos entre elas. Isso também pode ajudar você a identificar e corrigir possíveis áreas problemáticas antes que ocorra algum problema.

7.3 Diagrama de Sequência

O que é um diagrama de sequência em UML?

Diagrama de sequência é um tipo de diagrama de interação, pois descreve como, e em qual ordem, um grupo de objetos trabalha em conjunto. Estes diagramas são usados por desenvolvedores de software e profissionais de negócios para entender os requisitos de um novo sistema ou para documentar um processo existente. Diagramas de sequência são conhecidos como diagramas de eventos ou cenários de eventos.

Diagramas de sequência são úteis principalmente para visualizar interações ao longo do tempo em arquiteturas complexas, mantendo o fluxo compreensível tanto para partes interessadas técnicas quanto não técnicas.

Você também pode encontrar uma variante especializada chamada diagrama de sequência do sistema (DSS). Os diagramas de sequência do sistema, também conhecidos como DSSs, são na verdade um subtipo de diagramas de sequência em UML. Esses gráficos mostram detalhes de eventos gerados por atores de fora do sistema. Enquanto um diagrama de sequência padrão pode modelar a colaboração de vários objetos internos, um DSS normalmente trata o sistema como uma "caixa-preta" e se concentra no fluxo de eventos do ator externo ao sistema de um caso de uso específico.

Os benefícios de diagramas de sequência

Diagramas de sequência podem ser referências úteis para empresas e outras organizações. Experimente desenhar um diagrama de sequência para:

- Representa os detalhes de um caso de uso UML.
- Modelar a lógica de um processo, função ou operação sofisticado.
- Ver como objetos e componentes interagem uns com os outros para concluir um processo.
- Planejar e compreender a funcionalidade detalhada de um cenário existente ou futuro.

Se você estiver usando especificamente um diagrama de sequência do sistema (DSS), ele também pode ajudar a esclarecer como o sistema reage a eventos gerados externamente e quais operações esses eventos devem acionar. DSSs são

ideais para demonstrar quando e como as tarefas são concluídas em um sistema, principalmente porque essas tarefas são relacionadas a casos de uso.

Seguem alguns exemplos específicos de quando usar DSSs:

- De uma forma passo a passo, modelando operações do sistema em resposta a eventos

- Ao elaborar um estratagema para o principal cenário de sucesso de um determinado caso de uso e, em seguida, criar percursos alternativos.

- Ao identificar os principais eventos e operações do sistema, a fim de determinar estimativas realistas de recursos necessários

Casos de uso de diagramas de sequência

Os seguintes cenários são ideais para o uso de um diagrama de sequência:

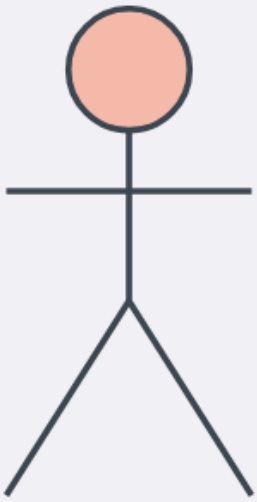
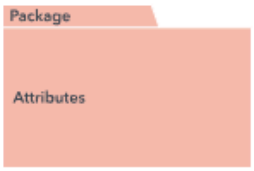
- **Cenário de uso:** um cenário de uso é um diagrama de como seu sistema poderia ser utilizado. É ótimo para certificar-se de que você compreendeu a lógica de cada cenário de uso do sistema.

- **Lógica do método:** assim como você usaria um diagrama de sequência UML para explorar a lógica de um caso de uso, você pode usá-lo para explorar a lógica de qualquer função, procedimento ou processo complexo.

- **Lógica de serviço:** se você acredita que serviços são métodos de alto nível usados por clientes, um diagrama de sequência é ideal para mapeá-lo.

Símbolos e componentes básicos

Símbolo	Nome	Descrição
	Símbolos de objetos	Representa uma classe ou um objeto em UML. O símbolo do objeto demonstra como um objeto se comportará no contexto do sistema. Atributos de classe não devem ser listados nesta forma.
	Caixa de ativação	Representa o tempo necessário para que um objeto conclua uma tarefa. Quanto mais tempo a tarefa levar, mais alongada a caixa de ativação se torna.

	Símbolo de ator	Mostra as entidades que interagem com, ou são externas ao sistema.
	Símbolo de pacote	Usada em notação UML 2.0 para conter elementos interativos do diagrama. Também conhecida como quadro, esta forma retangular tem um pequeno retângulo interno para rotular o diagrama.
	Símbolo de linha da vida	Representa a passagem do tempo, conforme estende-se para baixo. Esta linha vertical tracejada mostra os eventos sequenciais que ocorrem a um objeto durante o processo no gráfico. Linhas da vida podem começar com uma forma de retângulo rotulado ou um símbolo de ator.

	Símbolo de opção em loop	Usado para modelar cenários "se/então", ou seja, uma circunstância que ocorrerá apenas sob determinadas condições.
	Símbolo alternativo	Simboliza uma escolha (geralmente mutuamente exclusiva) entre duas ou mais sequências de mensagens. Para representar alternativas, use a forma de retângulo rotulada com uma linha tracejada em seu interior.

Nos diagramas de sequência do sistema (DSS), aparecem muitos desses mesmos elementos, mas eles costumam ser usados com um foco mais restrito nos atores externos, no limite do sistema e nos eventos trocados entre eles. A maioria dos elementos que descrevemos nos diagramas de casos de uso permanece em uso em todo o diagrama de sequência do sistema, incluindo:

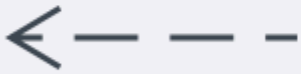
- **Objetos:** esta forma de caixa com um título sublinhado representa uma classe (ou objeto) em UML. Dentro de um DSS, esta forma modela o sistema como uma caixa preta (um sistema com funcionamento interno que não é imediatamente visível).

- **Atores:** representados por um boneco palito, atores são entidades que interagem com o sistema sem fazer parte dele.

- **Eventos:** os eventos do sistema que os atores geram dentro da sequência. Uma linha tracejada, conhecida como linha da vida, representa eventos em um DSS. Linhas de vida podem começar com uma forma de retângulo rotulado ou um símbolo de ator.

Símbolos comuns de mensagens

Use as seguintes setas e símbolos de mensagens para mostrar como as informações são transmitidas entre os objetos. Estes símbolos podem refletir o início e a execução de uma operação, ou o envio e a recepção de um sinal.

Símbolo	Nome	Descrição
	Símbolo de mensagens síncronas	Representado por uma linha sólida com uma ponta de seta sólida. Este símbolo é usado quando um remetente deve esperar por uma resposta a uma mensagem antes de poder continuar. O diagrama deve mostrar tanto a chamada quanto a resposta.
	Símbolo de mensagens assíncronas	Representado por uma linha sólida com uma ponta de seta em linha. Mensagens assíncronas não necessitam de uma resposta para que o remetente continue. Apenas a chamada deve ser incluída no diagrama.
	Símbolo de mensagens de retorno assíncronas	Representado por uma linha tracejada com uma ponta de seta em linha.
	Símbolo assíncrono de criar mensagem	Representado por uma linha tracejada com uma ponta de seta em linha. Esta mensagem cria um novo objeto.
	Símbolo de responder à mensagem	Representado por uma linha tracejada com uma ponta de seta em linha, estas mensagens são respostas às chamadas.

Um lembrete prático ao posicionar mensagens: os objetos podem se comunicar por meio do diagrama e não precisam estar adjacentes, desde que as setas conectem claramente as linhas de vida e as ativações corretas. Lembre-se de que qualquer objeto pode enviar ou receber uma mensagem de qualquer outro, portanto, eles não precisam estar lado a lado.

7. 4 Diagrama de Componentes

O que é diagrama de componentes UML?

O diagrama de componentes mostra o relacionamento entre diferentes componentes de um sistema. Para fins de UML 2.0, o termo "componente" refere-se a um módulo de classes que representa sistemas ou subsistemas independentes com capacidade de interagir com o restante do sistema.

Para isso, existe uma abordagem de desenvolvimento em torno de componentes: o desenvolvimento baseado em componentes (CBD). Nela, o diagrama de componentes identifica os diferentes componentes para que todo o sistema funcione corretamente.

O mais comum, em uma abordagem de programação OO, é o desenvolvedor sênior usar o diagrama de componentes para agrupar classes com base em um objetivo comum, para que o desenvolvedor e outras pessoas envolvidas analisem o projeto de desenvolvimento de software de forma generalizada.

Benefícios do diagrama de componentes

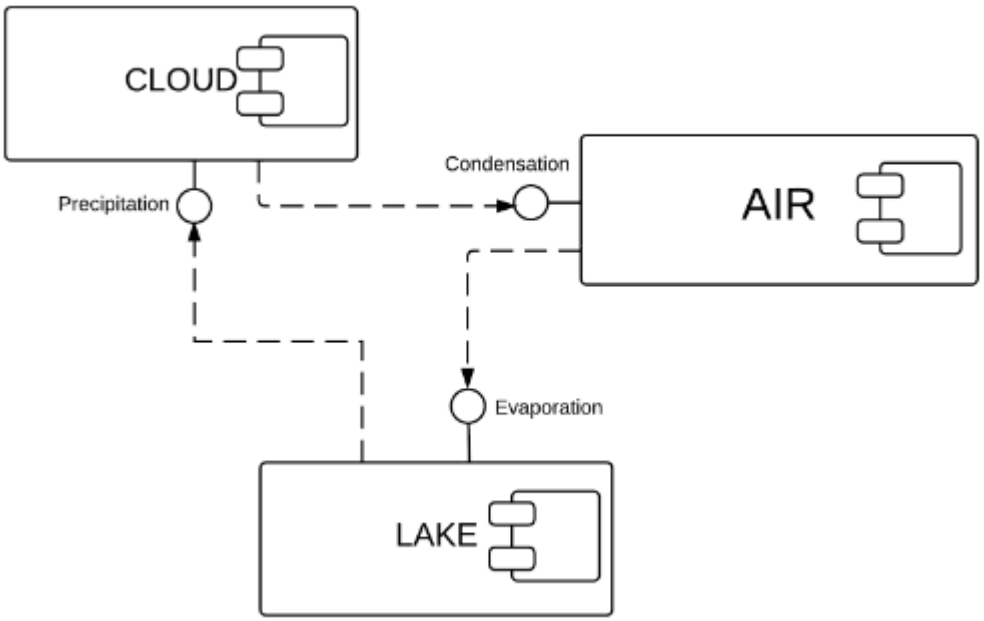
Embora o diagrama de componentes pareça complexo à primeira vista, ele é indispensável na criação do seu sistema. Com ele, sua equipe:

- Imagina a estrutura física do sistema.
- Presta atenção aos componentes do sistema e como eles se relacionam.
- Enfatiza o comportamento do serviço quanto à interface.

Como usar o diagrama de componentes

O diagrama de componentes em UML dá uma visão geral do sistema de software. Compreender o comportamento exato do serviço fornecido por cada software fará de você um desenvolvedor melhor. O diagrama de componentes descreve sistemas de software implementados em qualquer linguagem ou estilo de programação.

No diagrama de componentes, a UML determina que componentes e pacotes sejam conectados a linhas que representam conectores de montagem e de delegação.



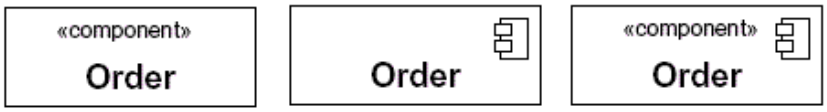
Formas e símbolos do diagrama de componentes

O diagrama de componentes pode ser simples, geral ou detalhado e complexo. De toda forma, recomendamos conhecer os símbolos UML apropriados. Veja abaixo os tipos de formas mais comuns ao ler e criar um diagrama de componentes:

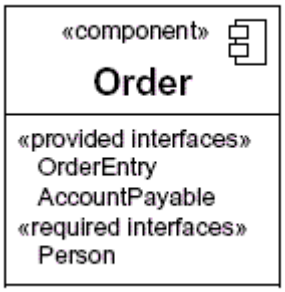
	Símbolo de componente	Entidade necessária para executar uma função de estereótipo. O componente fornece e consome comportamento em interfaces ou por meio de outros componentes. Considere os componentes como um tipo de classe. Na UML 1.0, o componente tem forma de bloco retangular, com dois retângulos menores saindo pela lateral. Na UML 2.0, o componente tem forma de bloco retangular com uma pequena imagem da forma antiga do diagrama de componentes.
	Símbolo de nó	Representa objetos de hardware ou software, de nível superior aos componentes.
	Símbolo de interface	Mostra entradas ou materiais que o componente recebe ou fornece. A interface pode ser representada por notas ou símbolos de texto, como formato de pirulito, soquete e "bola-e-soquete".

	Símbolo de porta	Especifica um ponto de interação separado entre o componente e o ambiente. O símbolo da porta é um pequeno quadrado.
	Símbolo de pacote	Agrupa vários elementos do sistema e é representado por pastas de arquivos no Lucidchart. Assim como as pastas de arquivos agrupam várias folhas, o pacote pode ser desenhado em torno de vários componentes.
	Símbolo de nota	Com ele, os desenvolvedores fixam uma meta-análise no diagrama de componentes.
	Símbolo de dependência	Mostra que uma parte do sistema depende de outra. A dependência é representada por linhas tracejadas que vinculam um componente (ou elemento) a outro.

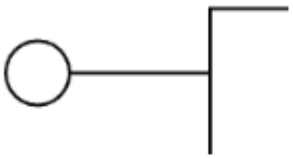
Como usar formas e símbolos de componentes



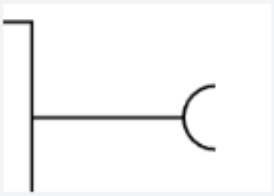
Existem três maneiras comuns de criar o compartimento de nome do componente. É sempre necessário incluir o texto do componente entre aspas angulares e/ou o logotipo do componente. Essa diferença é importante, pois o retângulo com apenas um nome dentro representa classificadores (elementos de classe).



Assim como na notação de classe, o componente também tem um espaço para listar interfaces, semelhante a como atributos e métodos à notação de classe são adicionados. A interface representa os locais onde os grupos de classes no componente se comunicam com outros componentes do sistema. Outra maneira de representar a interface é estender um símbolo da caixa de componentes. Veja um rápido resumo dos símbolos mais usados.



Interface fornecida: uma linha reta a partir da caixa de componentes e com um círculo anexado. Esse símbolo representa a interface na qual um componente produz informações usadas pela interface necessária de outro componente.



Interface necessária: uma linha reta da caixa de componentes e com um semicírculo anexado (também representado como uma seta tracejada com uma seta aberta). Esse símbolo representa a interface na qual um componente requer informações para executar a função correta.

Na UML, o diagrama de componentes representa visualmente como os componentes de um sistema de software se relacionam. Para criar um, use a biblioteca de formas de diagramas de componentes personalizada do Lucidchart. O diagrama de componentes deve mostrar:

- O escopo do sistema
- A estrutura geral do sistema de software
- As metas que o sistema ajuda entidades humanas ou não humanas (conhecidas como atores) a atingir

7.5 Diagrama de Implementação

O que é um diagrama de implementação?

No contexto da Linguagem de modelagem unificada (UML), um diagrama de implementação faz parte da família de diagramação estrutural pois descreve um aspecto do sistema em si. Neste caso, o diagrama de implementação descreve a implementação física de informações geradas pelo programa de software em componentes de hardware. A informação gerada pelo software é chamada de artefato. Não se deve confundir este com o uso do mesmo termo em outras abordagens de modelagem, como BPMN.

Os diagramas de implementação são feitos de várias formas de UML. As caixas tridimensionais conhecidas como nós representam os elementos básicos de software ou hardware, ou nós do sistema. As linhas de nó a nó indicam

relacionamentos e as formas menores contidas dentro das caixas representam os artefatos de software empregados. Por descreverem hardware, os diagramas de implantação são únicos no universo UML.

Aplicações do diagrama de implementação

Os diagramas de implementação possuem várias aplicações de grande valor. Eles podem ser usados para:

- Mostrar quais elementos de software são empregados e quais elementos de hardware.
- Ilustrar o processamento do tempo de execução do hardware.
- Propiciar uma visão da topologia do sistema do hardware.

Diretrizes do diagrama de implementação

. Antes de começar a criar diagramas, considere as seguintes perguntas:

- Você identificou o escopo de seu sistema? Por exemplo, é preciso saber se está diagramando um aplicativo único ou diagramando a implementação em toda uma rede de computadores.
- Quais as limitações de seu hardware físico? Com quais sistemas legados você precisará interagir? Certifique-se de que conhece o software em operação e os protocolos com os quais trabalhará e qual sistema de monitoração será adotado.
- Qual arquitetura de distribuição você está usando? É necessário saber quantos níveis seu aplicativo terá e em qual aplicativo você vai implantá-lo.
- Você possui todos os nódulos necessários? Você sabe como eles todos estão conectados?
- Você sabe quais componentes estão em quais nódulos?

Depois de responder a essas perguntas, pode ser útil elaborar um inventário simples dos nós que você planeja mostrar e como eles se relacionam, e depois converter essa lista em notação UML.

Elementos do diagrama de implementação

Diagramas de implementação são constituídos por uma variedade de formas. Esta lista oferece uma visão geral dos elementos básicos que podem ser encontrados. É possível ver a maioria destes itens ilustrados na imagem abaixo

- **Artefato:** um produto desenvolvido pelo software, simbolizado por um retângulo com o nome e a palavra “artefato” entre setas duplas.

- **Associação:** uma linha que indica uma mensagem ou outro tipo de comunicação entre nós.

- **Componente:** um retângulo com duas abas que indica um elemento de software.

- **Dependência:** uma linha tracejada finalizada em uma seta, indicando que um nó ou componente é dependente de um outro.

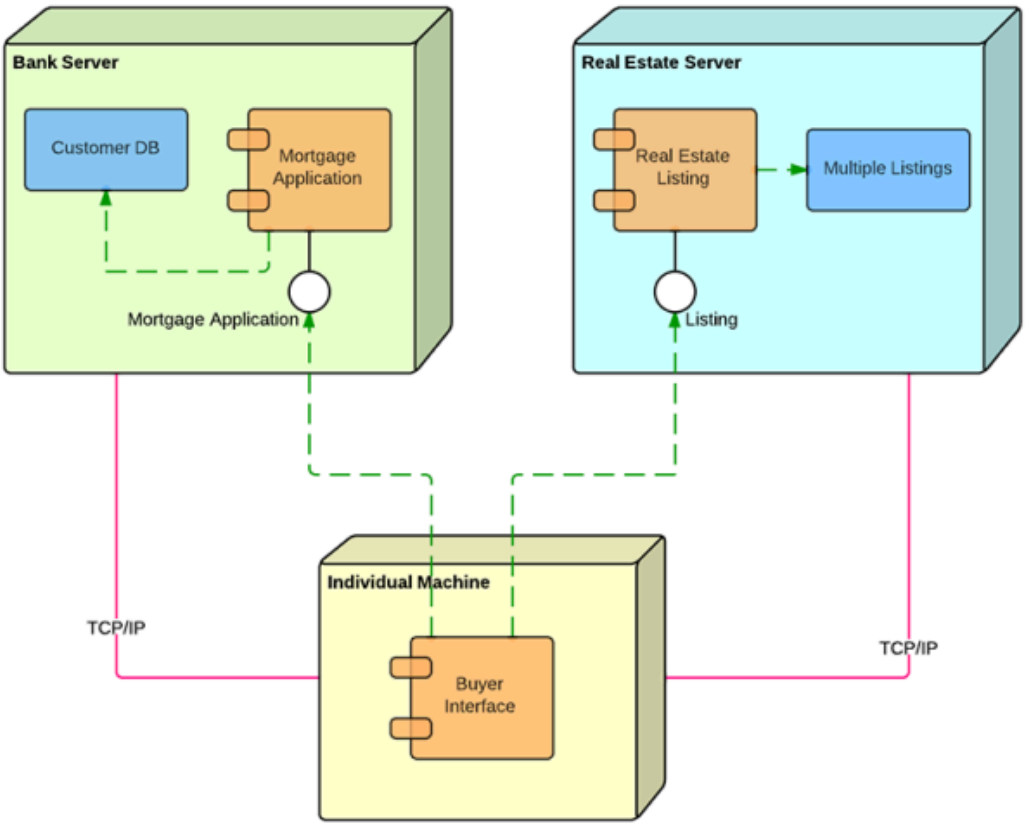
- **Interface:** um círculo que indica uma relação contratual.São aqueles objetos que notam que a interface deve realizar algum tipo de obrigação.

- **Nó:** um objeto de hardware ou software representado por uma caixa tridimensional.

- **Nó como contêiner:** um nó que contém outro nó em seu interior, tal como no exemplo abaixo onde os nós contêm componentes.

- **Estereótipo:** um dispositivo contido dentro do nó, apresentado na parte superior do nó, com o nome entre setas duplas.

Na prática, você geralmente decide primeiro o objetivo do diagrama (por exemplo, topologia x processamento em tempo de execução) e, em seguida, escolhe quais desses elementos são necessários para expressar esse objetivo com clareza.



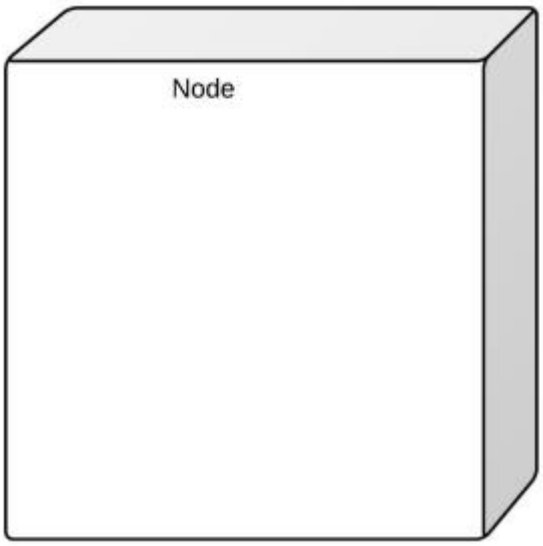
Símbolos e notação do diagrama de implementação

Use estas formas para criar diagramas de implementação UML.

Nódulos (Nós)

Existem dois tipos de nós em um diagrama de implementação: nós de dispositivos e nós de ambiente de execução. Nós de dispositivos são recursos de computação com capacidades de processamento e execução de programas. Alguns exemplos de nós de dispositivos são PCs, laptops ou celulares.

Um nó de ambiente de execução, ou EEN, é qualquer sistema de computação que resida dentro de um nó de dispositivo. Pode ser um sistema operacional, um JVM ou outro recipiente web.



Base de dados

Bancos de dados representam qualquer dado armazenado pelo sistema implementado. Em alguns casos, o banco de dados é representado como um outro nó, no entanto, às vezes você verá esta forma representando um banco de dados.

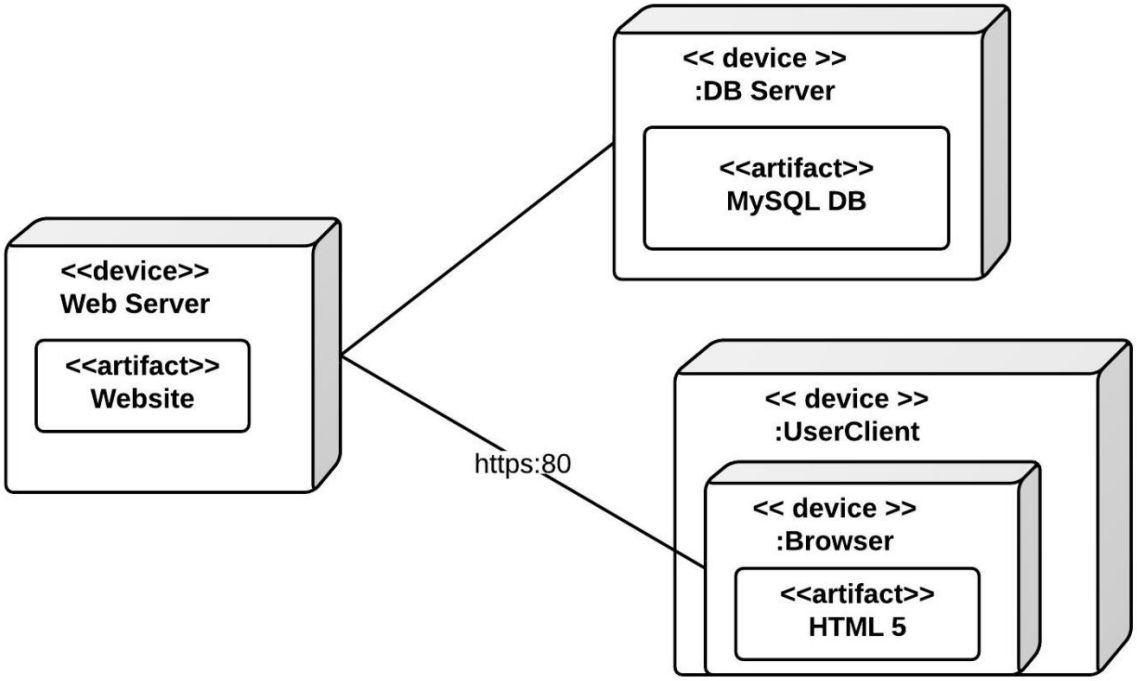


Outras formas

- Percurso da comunicação: uma linha reta que representa a comunicação entre dois nódulos de aparelho.
- Artefatos: Uma caixa com o cabeçalho "<>" e depois o nome do arquivo.
- Pacote: uma caixa em forma de arquivo que agrupa todos os nós de dispositivos para integrar toda a implementação.
- Componente: uma entidade necessária para executar uma função de estereótipo.

Posicionando os símbolos, lembre-se de que diagramas de implementação podem variar do simples ao muito detalhado. Por exemplo, você pode mostrar um nó aninhado dentro de outro para refletir um padrão de implantação específico.

Exemplo de diagrama de implementação



Este exemplo mostra um diagrama básico de implementação. Há um servidor web, um servidor de banco de dados e o aparelho de onde o usuário visualiza o site. É possível acrescentar uma maior complexidade mostrando as diferentes partes do servidor web e como o Javascript funciona no UserClient, mas este exemplo mostra um pouco como é a implementação ao usarmos a notação UML.

REFERÊNCIAS

ALURA. O que é metodologia ágil. [S. l.], [s. d.]. Disponível em: https://www.alura.com.br/artigos/o-que-e-metodologia-agil. Acesso em: 9 set. 2026.

ANTHROPIC. Claude. [Modelo de linguagem]. São Francisco: Anthropic, 2026. Disponível em: https://claude.ai. Acesso em: 9 set. 2026.

BASTOS JÚNIOR, Paulo Roberto O. *Elicitação de requisitos de software através da utilização de questionários*. 2005. Dissertação (Mestrado) – Rio de Janeiro, 2005.

BELGAMO, Anderson; MARTINS, Luiz E. G. Estudo comparativo sobre as técnicas de elicitação de requisitos do software. São Paulo, 2000.

DUARTE, Renato. *Quatro pilares da orientação a objetos e encapsulamento*. São Caetano do Sul: Universidade Municipal de São Caetano do Sul (USCS), 2023. Slides de aula.

ESPÍNDOLA, Rafael Senti. *Um estudo das técnicas de elicitação de requisitos de software*. 2008. Trabalho de Conclusão de Curso – São Paulo, 2008.

GOOGLE. Gemini. [Modelo de linguagem]. Mountain View: Google, 2026. Disponível em: https://gemini.google.com. Acesso em: 9 set. 2026.

LUCID. Tutorial de diagrama de atividades UML. [S. l.], [s. d.]. Disponível em: https://lucid.co/pt/diagrama/uml/tutorial-de-diagrama-de-atividades. Acesso em: 9 set. 2026.

LUCID. Tutorial de diagrama de classes. [S. l.], [s. d.]. Disponível em: https://lucid.co/pt/diagrama/uml/tutorial-de-diagrama-de-classes. Acesso em: 9 set. 2026.

LUCID. Tutorial de diagrama de sequência. [S. l.], [s. d.]. Disponível em: https://lucid.co/pt/diagrama/uml/tutorial-de-diagrama-de-sequencias. Acesso em: 9 set. 2026.

LUCID. Tutorial sobre diagrama de caso de uso. [S. l.], [s. d.]. Disponível em: https://lucid.co/pt/diagrama/uml/tutorial-de-diagrama-de-caso-de-uso. Acesso em: 9 set. 2026.

MORAES, Janaína Bedani Dixon. Engenharia de software 2: técnicas para levantamento de requisitos. [S. l.], [s. d.]. Disponível em: https://www.devmedia.com.br/engenharia-de-software-2-tecnicas-para-levantamento-de-requisitos/9151. Acesso em: 9 set. 2026.

MORALES, Marcelo. *Classes e objetos*. São Caetano do Sul: Universidade Municipal de São Caetano do Sul (USCS), 2023. Slides de aula.

OPENAI. ChatGPT. [Modelo de linguagem]. São Francisco: OpenAI, 2026. Disponível em: https://chatgpt.com. Acesso em: 9 set. 2026.

PERRELLA, Fabricio. *Conceitos e evolução da engenharia de software*. São Caetano do Sul: Universidade Municipal de São Caetano do Sul (USCS), 2024. Slides de aula.

PERRELLA, Fabricio. *Modelagem de software*. São Caetano do Sul: Universidade Municipal de São Caetano do Sul (USCS), 2024. Slides de aula.

PERRELLA, Fabricio. *Princípios de engenharia de requisitos*. São Caetano do Sul: Universidade Municipal de São Caetano do Sul (USCS), 2024. Slides de aula.

PERRELLA, Fabricio. *Processos de software*. São Caetano do Sul: Universidade Municipal de São Caetano do Sul (USCS), 2024. Slides de aula.

PERRELLA, Fabricio. *UML completo*. São Caetano do Sul: Universidade Municipal de São Caetano do Sul (USCS), 2024. Slides de aula.

PEREIRA, Thiago. Elicitação de requisitos e suas técnicas: parte 02. 2010. Disponível em: https://imasters.com.br/artigo/18365/gerencia-de-projetos/elicitacao-de-requisitos-e-suas-tecnicas-parte-02/. Acesso em: 9 set. 2026.

PINTO, Rochelly Sirremes; FONTENELLE, Maria Aridenise Macena. Desdobramento da função qualidade (QFD) no processo de desenvolvimento de produtos: uma aplicação prática. 2013. Disponível em: http://www.abepro.org.br/biblioteca/enegep2013_TN_STP_181_033_22774.pdf. Acesso em: 9 set. 2026.

PRESSMAN, Roger S. *Engenharia de software: uma abordagem profissional*. 7. ed. Porto Alegre: AMGH, 2011.

PUCRS. Princípios do PMBOK. *Blog PUCRS Online*, [s. d.]. Disponível em: https://online.pucrs.br/blog/principios-do-pmbok. Acesso em: 9 set. 2026.

QUINTANS, Maria Ludovina. *Um estudo exploratório sobre o conhecimento e utilização de técnicas de elicitação de requisitos em empresas de software*. 2009/2010. Dissertação de Mestrado (Qualificação).

ROCKETSEAT. Programação orientada a objetos: princípios e boas práticas. *Blog Rocketseat*, [s. d.]. Disponível em: https://www.rocketseat.com.br/blog/artigos/post/programacao-orientada-a-objetos-principios-e-boas-praticas. Acesso em: 9 set. 2026.

ROSEMBERG, Carlos; SCHILLING, Albert; BASTOS, Cristianne; ARARIPE, Rodrigo. Elicitação de requisitos e design participativo através de protótipos de baixa

fidelidade: um estudo de caso. [S. l.], [s. d.]. Disponível em: http://www.infobrasil.inf.br/userfiles/Elicita%C3%A7%C3%A3o_de_Requisitos_e_Design_Participativo_atrav%C3%A9s_de_Prot%C3%B3tipos_de_baixa_Fidelidade.pdf. Acesso em: 9 set. 2026.

SOMMERVILLE, Ian. *Engenharia de software*. 8. ed. São Paulo: Addison Wesley, 2007.

TOTVS. Metodologia ágil. *Blog TOTVS*, [s. d.]. Disponível em: https://www.totvs.com/blog/negocios/metodologia-agil/. Acesso em: 9 set. 2026.

TOTVS. PMBOK. *Blog TOTVS*, [s. d.]. Disponível em: https://www.totvs.com/blog/negocios/pmbok/. Acesso em: 9 set. 2026.

OMG. Introdução à linguagem de modelagem unificada (UML). [S. l.], [s. d.]. Disponível em: https://www.omg.org/uml/. Acesso em: 9 set. 2026.

ESTRATÉGIA CONCURSOS. Técnicas de elicitação de requisitos: EBSERH. [S. l.], [s. d.]. Disponível em: https://www.estrategiaconcursos.com.br/blog/tecnicas-elicitacao-requisitos-ebserh/. Acesso em: 9 set. 2026.

DEUTNERG. Arquitetura backend: a arquitetura em camadas. *Medium*, [s. d.]. Disponível em: https://medium.com/@deutnerg/arquitetura-backend-a-arquitetura-em-camadas-01368fc13797. Acesso em: 9 set. 2026.

DEVMEDIA. Os 4 pilares da programação orientada a objetos. [S. l.], [s. d.]. Disponível em: https://www.devmedia.com.br/os-4-pilares-da-programacao-orientada-a-objetos/9264. Acesso em: 9 set. 2026.

EXCALIDRAW. Diagrama de sequência. [S. l.], [s. d.]. Disponível em: https://plus.excalidraw.com/use-cases/sequence-diagram#activation-boxes. Acesso em: 9 set. 2026.